

Redes Neurais Artificiais:

Redes de Múltiplas Camadas (MLP)

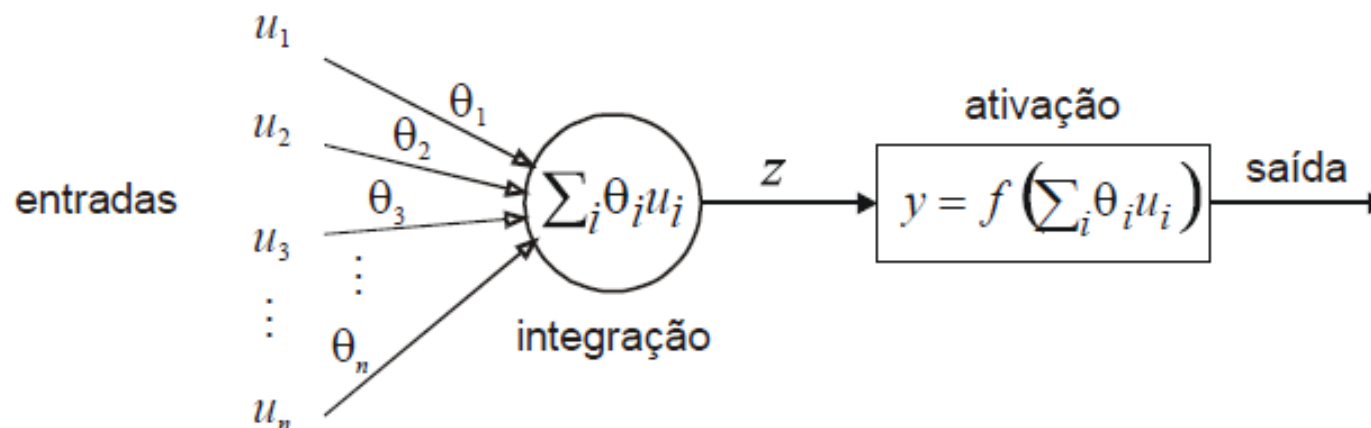
Edimilson B. dos Santos

Modelo de Neurônio

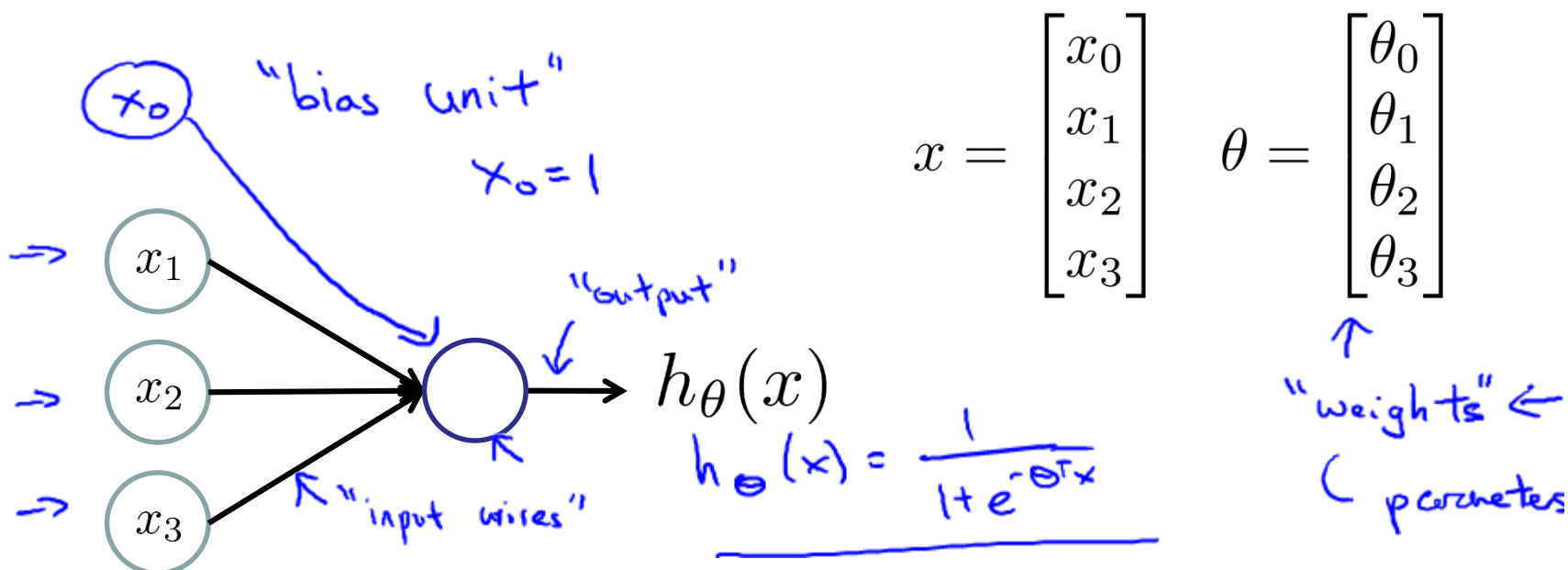
O neurônio de McCulloch-Pitt é um modelo matemático do neurônio biológico, onde a saída y é calculada a partir das entradas u_i como:

$$y = f(z) = f\left(\sum_i \theta_i u_i\right)$$

onde os parâmetros θ_i representam os pesos das conexões sinápticas; o valor z é chamado ativação e f é a função de ativação.

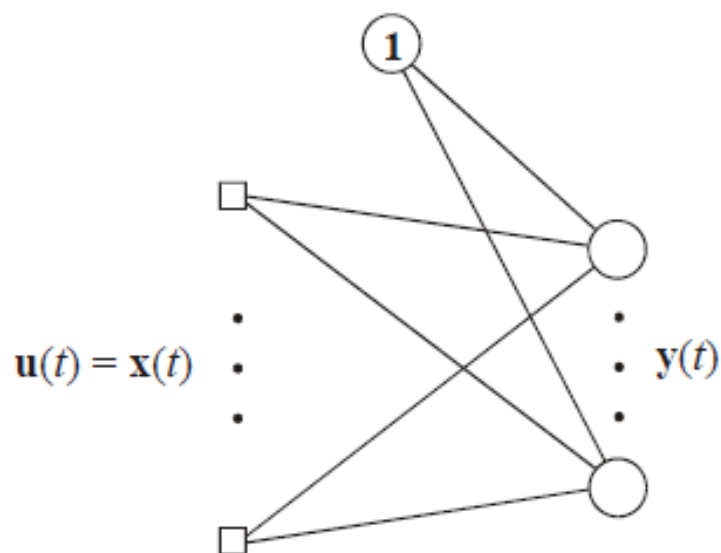


Modelo de Neurônio

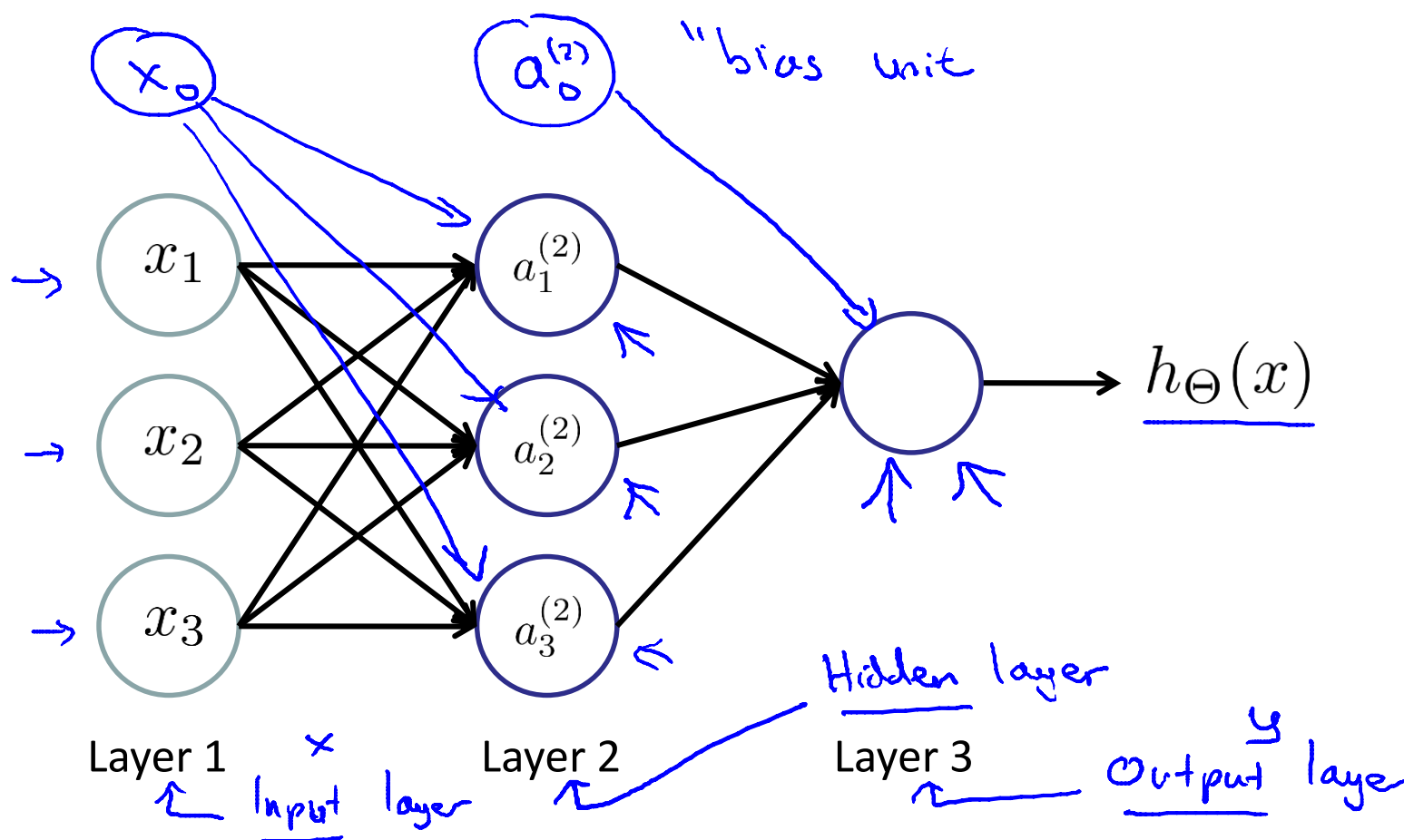


O Modelo em Camadas Simples

A topologia de RNA mais simples que pode ser obtida com o modelo de neurônio McCulloch-Pitt é a topologia em camada simples, onde as entradas estão conectadas diretamente às unidades de saída.

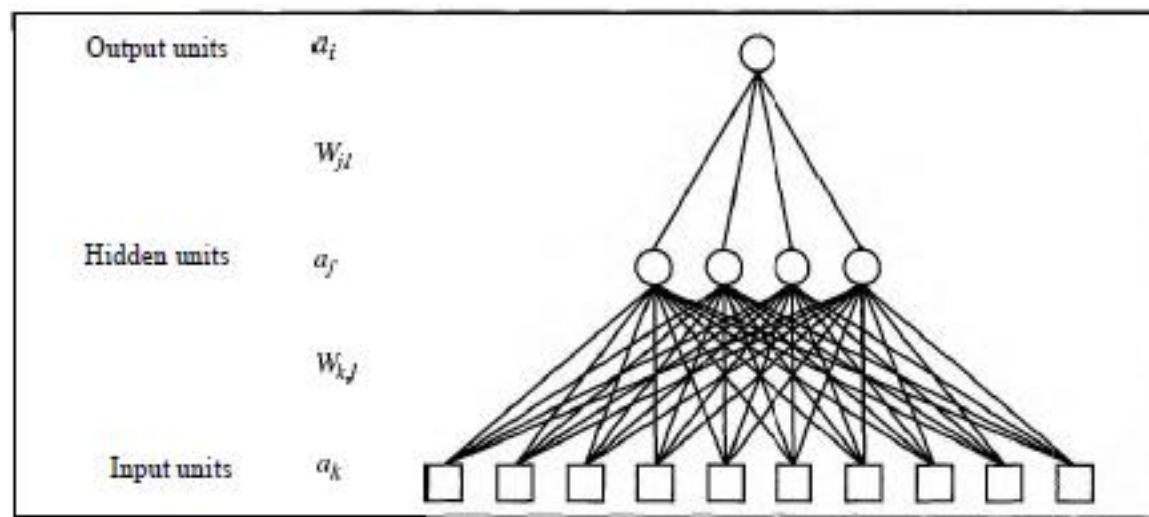


O Modelo em Camadas Múltiplas



O Modelo em Camadas Múltiplas

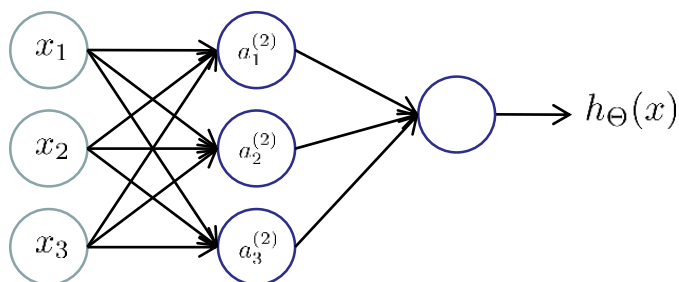
- As redes de múltiplas camadas (MLP) conectam os vários neurônios de uma camada com neurônios da camada seguinte.
- O caso mais comum envolve uma única camada oculta.



O Modelo em Camadas Múltiplas

- Adicionar camadas ocultas aumenta o espaço de hipóteses que a rede pode representar.
- Com uma camada oculta suficientemente grande, é possível representar qualquer função contínua das entradas.
- Infelizmente, para qualquer estrutura de rede específica, é mais difícil caracterizar exatamente que funções podem ser representadas e que funções não podem ser representadas.
- O problema de escolher com antecedência o número correto de unidades ocultas ainda não está bem compreendido.

O Modelo em Camadas Múltiplas



$a_i^{(j)}$ = “ativação” da unidade i na camada j

$\Theta^{(j)}$ = Matriz de pesos controlando o mapeamento da função da camada j para camada $j + 1$

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

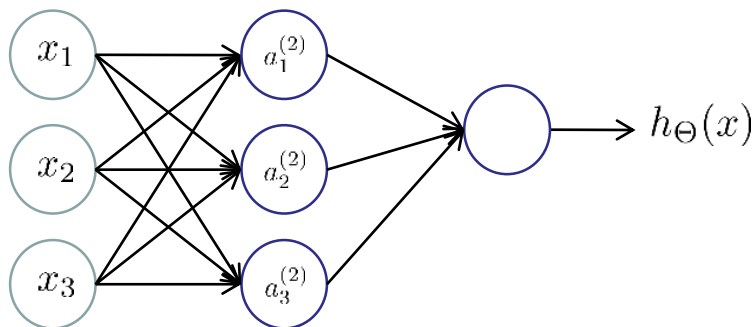
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

Se a rede tem s_j unidades na camada j , s_{j+1} unidades na camada $j + 1$, então $\Theta^{(j)}$ será de dimensão $s_{j+1} \times (s_j + 1)$.

O Modelo em Camadas Múltiplas



$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

$$z^{(2)} = \Theta^{(1)} x$$

$$a^{(2)} = g(z^{(2)})$$

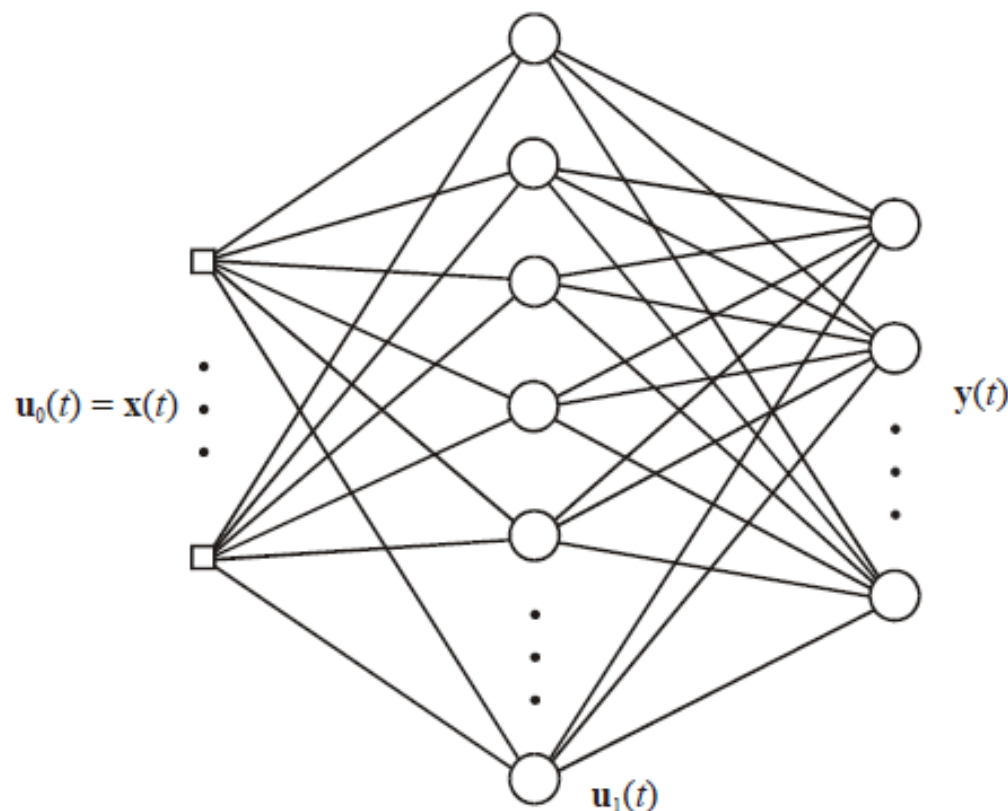
Add $a_0^{(2)} = 1.$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$

$$h_{\Theta}(x) = a^{(3)} = g(z^{(3)})$$

O Modelo em Camadas Múltiplas

Considere a rede MLP $p:n:m$ com p unidades na camada de entrada, n unidades na camada intermediária e m unidades na camada de saída



O Modelo em Camadas Múltiplas

A entrada da rede é dada pelo vetor de observações:

$$\mathbf{x}(t) = (x_1(t), \dots, x_p(t))$$

O resultado do processamento da rede neural é o vetor de saídas:

$$\mathbf{y}(t) = (y_1(t), \dots, y_m(t))$$

O processamento é realizado por cada camada:

$$\mathbf{u}(t) = \varphi_1(\mathbf{x}(t)): \text{primeira camada}$$

$$\mathbf{y}(t) = \varphi_2(\mathbf{u}(t)): \text{segunda camada}$$

O Modelo em Camadas Múltiplas

A saída da camada de entrada é uma cópia da entrada da rede:

$$\mathbf{u}_0(t) = (u_1^0(t), \dots, u_p^0(t)) = \mathbf{x}(t) = (x_1(t), \dots, x_p(t))$$

Cada unidade da camada de intermediária é calculada em função da camada anterior (de entrada) como:

$$u_j^1(t) = f(z_j^1(t)) = f\left(\sum_i \theta_{ij}^0 u_i^0(t) + \beta_j^0\right)$$

Em notação vetorial:

$$\mathbf{u}_1(t) = \mathbf{f}(\mathbf{z}_1(t)) = \mathbf{f}(\mathbf{u}_0(t)\mathbf{\Theta}_0 + \mathbf{\beta}_0)$$

O Modelo em Camadas Múltiplas

O cálculo da camada de intermediária é obtido pela seguinte equação vetorial:

$$\mathbf{u}_1(t) = \mathbf{f}(\mathbf{z}_1(t)) = \mathbf{f}(\mathbf{u}_0(t) \cdot \boldsymbol{\Theta}_0 + \boldsymbol{\beta}_0)$$

$$\mathbf{z}_1(t) = \mathbf{u}_0(t) \cdot \boldsymbol{\Theta}_0 + \boldsymbol{\beta}_0 \Rightarrow$$

$$\Rightarrow \begin{bmatrix} z_1^1(t) & \dots & z_n^1(t) \end{bmatrix} = \begin{bmatrix} u_1^0(t) & \dots & u_p^0(t) \end{bmatrix} \begin{bmatrix} \theta_{11}^0 & \dots & \theta_{1n}^0 \\ \vdots & \ddots & \vdots \\ \theta_{p1}^0 & \dots & \theta_{pn}^0 \end{bmatrix} + \begin{bmatrix} \beta_1^0(t) & \dots & \beta_n^0(t) \end{bmatrix}$$

$$\mathbf{u}_1(t) = \mathbf{f}(\mathbf{z}_1(t)) = \begin{bmatrix} f(z_1^1(t)) & \dots & f(z_n^1(t)) \end{bmatrix}$$

O Modelo em Camadas Múltiplas

Da mesma forma, a camada de saída é calculada em função da camada intermediária:

$$\mathbf{y}(t) = \mathbf{u}_2(t) = \mathbf{f}(\mathbf{z}_2(t)) = \mathbf{f}(\mathbf{u}_1(t) \cdot \Theta_1 + \beta_1)$$

$$\mathbf{z}_2(t) = \mathbf{u}_1(t) \cdot \Theta_1 + \beta_1 \Rightarrow$$

$$\Rightarrow \begin{bmatrix} z_1^2(t) & \dots & z_m^2(t) \end{bmatrix} = \begin{bmatrix} u_1^1(t) & \dots & u_n^1(t) \end{bmatrix} \begin{bmatrix} \theta_{11}^1 & \dots & \theta_{1m}^1 \\ \vdots & \ddots & \vdots \\ \theta_{n1}^1 & \dots & \theta_{nm}^1 \end{bmatrix} + \begin{bmatrix} \beta_1^1(t) & \dots & \beta_m^1(t) \end{bmatrix}$$

$$\mathbf{y}(t) = \mathbf{f}(\mathbf{z}_2(t)) = \begin{bmatrix} f(z_1^2(t)) & \dots & f(z_m^2(t)) \end{bmatrix}$$

Para uma estrutura geral com qualquer número de camadas, a saída de uma camada é calculada em função da saída da camada anterior

$$\mathbf{u}_k(t) = \mathbf{f}(\mathbf{z}_k(t)) = \mathbf{f}(\mathbf{u}_{k-1}(t) \cdot \Theta_{k-1} + \beta_{k-1})$$

O Modelo em Camadas Múltiplas: Topologia

A topologia da rede MLP $p:n:m$ com p unidades na camada de entrada, n unidades na camada intermediária e m unidades na camada de saída

- **Em problemas de classificação:**

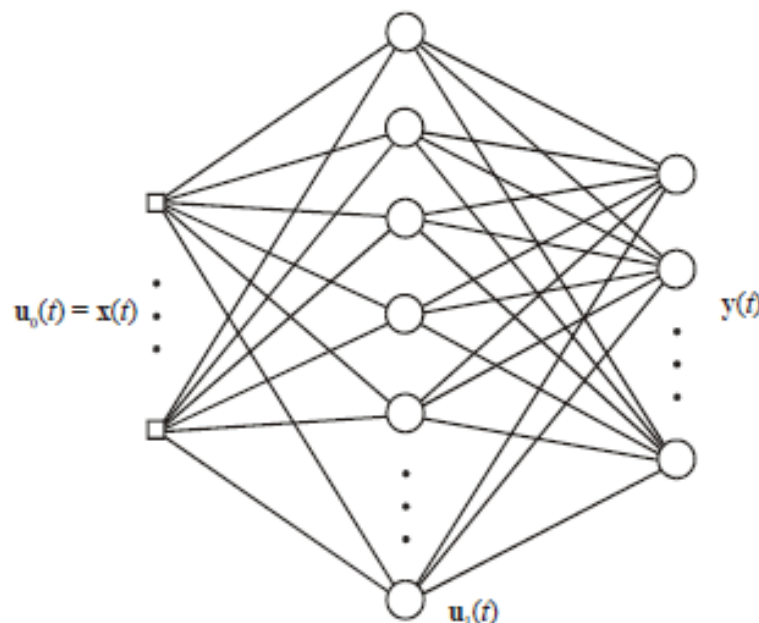
p é o número de variáveis de entrada

m é o número de classes

- **Em problemas de aproximação:**

p é o número de variáveis de entrada

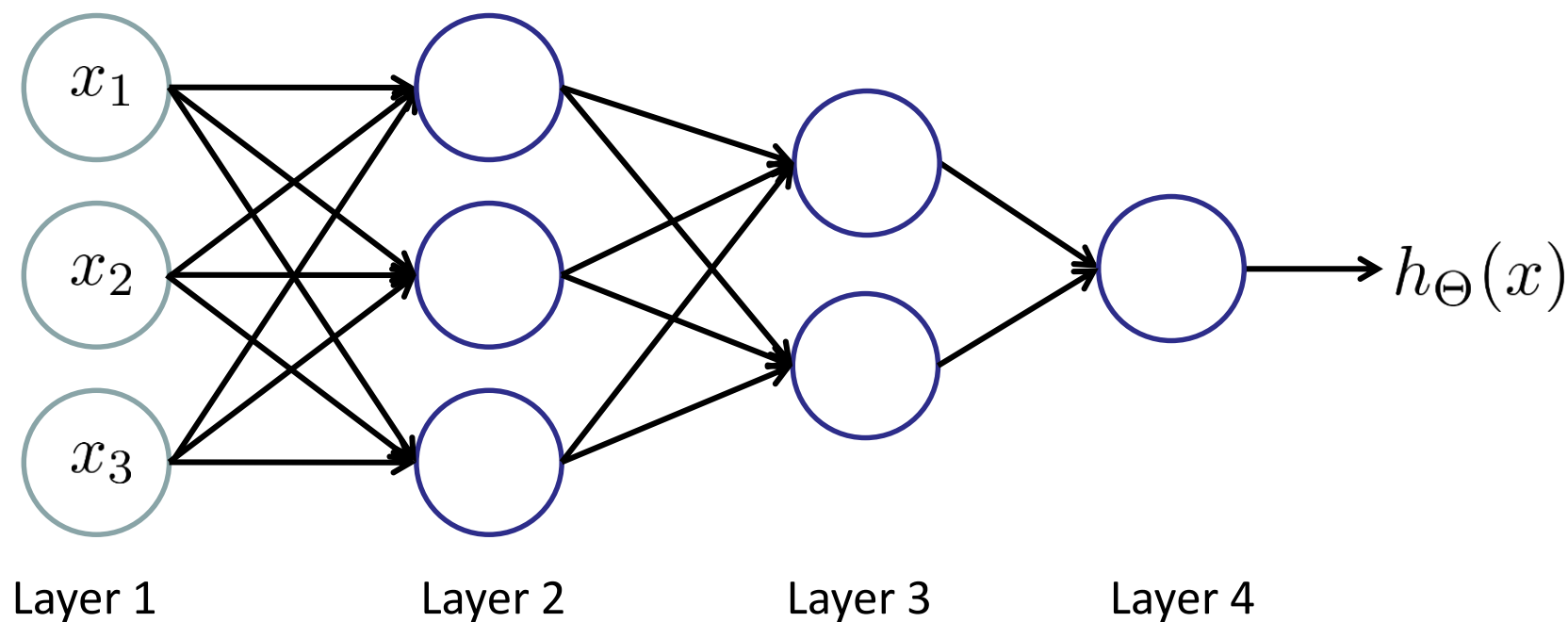
m é o número de variáveis de saída



O numero de camadas e o número de unidades na(s) camada(s) intermediárias definem a complexidade da rede

O Modelo em Camadas Múltiplas: Topologia

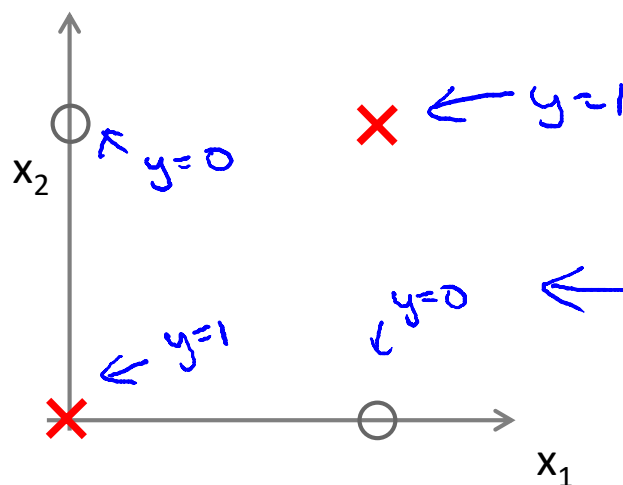
Outras arquiteturas de rede



O Modelo em Camadas Múltiplas

Exemplo de classificação não-linear: XOR/XNOR

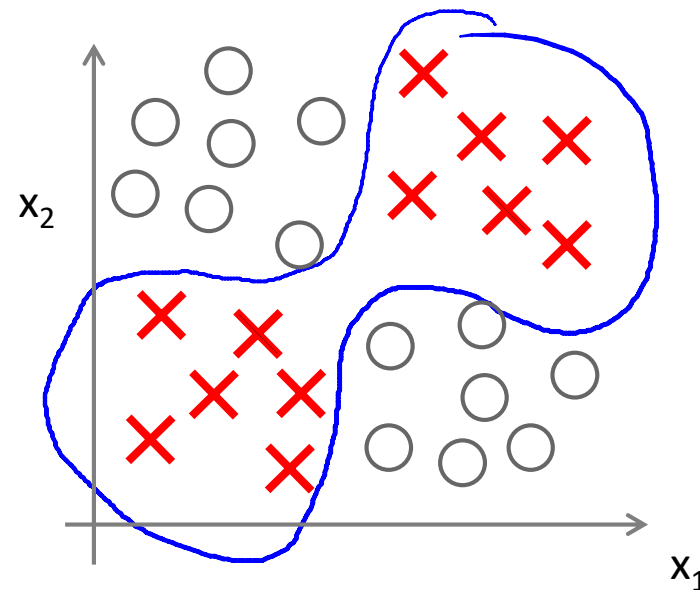
→ x_1, x_2 são binários (0 ou 1).



$$y = x_1 \text{ XOR } x_2$$

$$\text{ } \rightarrow x_1 \text{ XNOR } x_2 \leftarrow$$

$$\text{ } \rightarrow \text{NOT } (x_1 \text{ XOR } x_2)$$

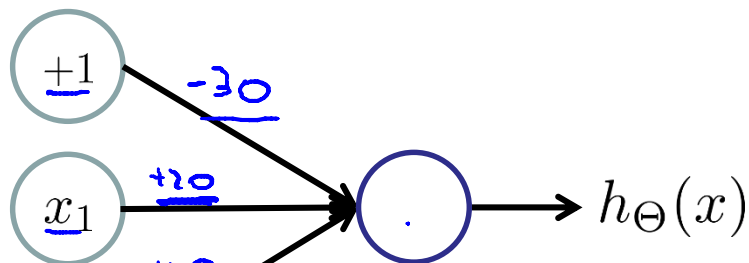


O Modelo em Camadas Múltiplas

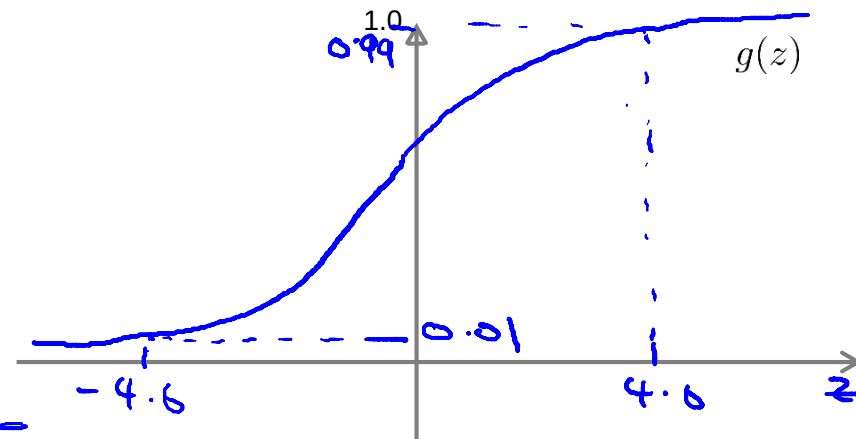
Exemplo de classificação não-linear: XOR/XNOR

Exemplo Simples: AND

- $x_1, x_2 \in \{0, 1\}$
- $y = x_1 \text{ AND } x_2$



$$\rightarrow h_{\Theta}(x) = g\left(\underbrace{-30}_{\omega_{10}^{(1)}} + \underbrace{20}_{\omega_{11}^{(1)}}x_1 + \underbrace{20}_{\omega_{12}^{(1)}}x_2\right)$$



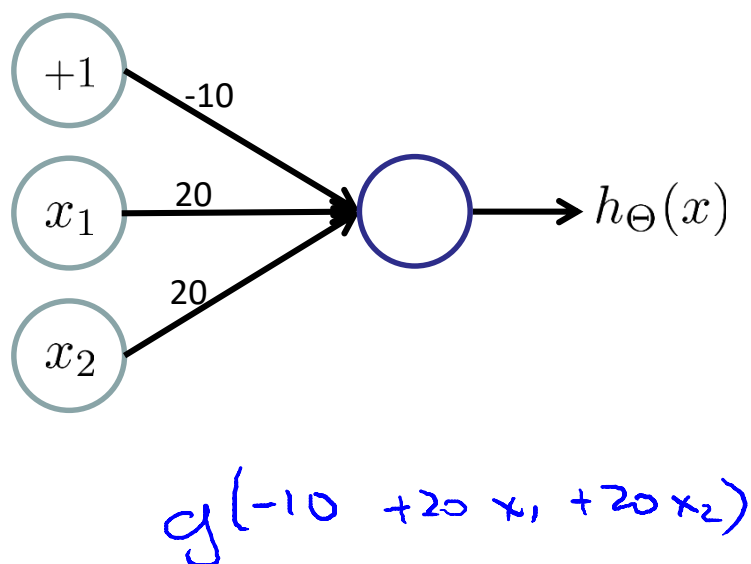
x_1	x_2	$h_{\Theta}(x)$
0	0	$g(-30) \approx 0$
→ 0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
→ 1	1	$g(10) \approx 1$

$h_{\Theta}(x) \approx x_1 \text{ AND } x_2$

O Modelo em Camadas Múltiplas

Exemplo de classificação não-linear: XOR/XNOR

Exemplo: OR



x_1	x_2	$h_{\Theta}(x)$
0	0	$g(-10) \approx 0$
0	1	$g(10) \approx 1$
1	0	≈ 1
1	1	≈ 1

O Modelo em Camadas Múltiplas

Exemplo de classificação não-linear: XOR/XNOR

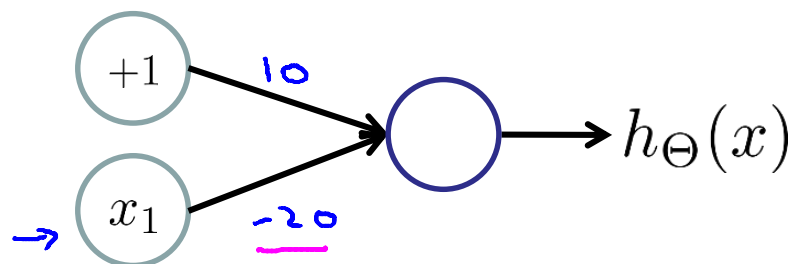
$\{0, 1\}$

$\rightarrow x_1 \text{ AND } x_2$

$\rightarrow x_1 \text{ OR } x_2$

Negation:

NOT x_1



x_1	$h_{\Theta}(x)$
0	$g(10) \approx 1$
1	$g(-10) \approx 0$

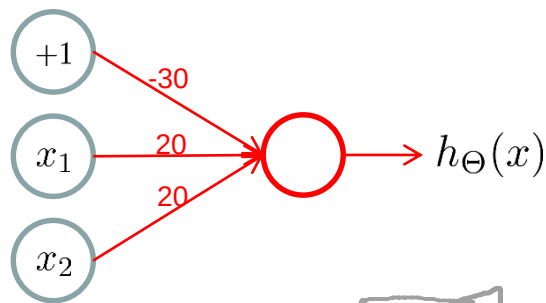
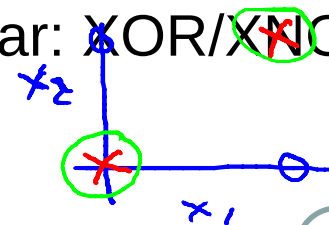
$$h_{\Theta}(x) = g(\underline{10 - 20x_1})$$

$\rightarrow (\text{NOT } x_1) \text{ AND } (\text{NOT } x_2)$
 $= 1$ if and only if
 $\rightarrow x_1 = x_2 = 0$

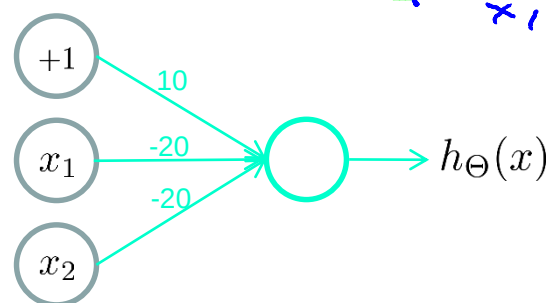
O Modelo em Camadas Múltiplas

Exemplo de classificação não-linear: XOR/XNOR

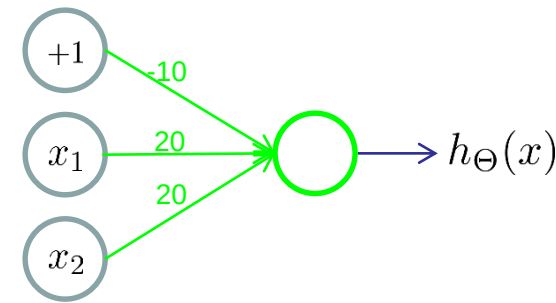
Putting it together: x_1 XNOR x_2



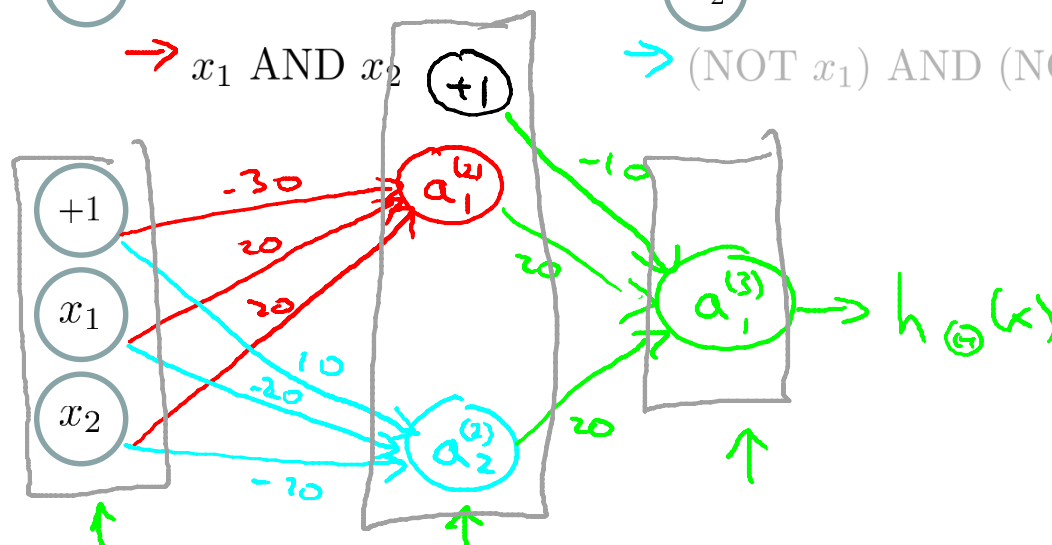
$\rightarrow x_1 \text{ AND } x_2$



$\rightarrow (\text{NOT } x_1) \text{ AND } (\text{NOT } x_2)$



$\rightarrow x_1 \text{ OR } x_2$



x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_{\Theta}(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

O Modelo em Camadas Múltiplas

Exemplo: Múltiplas unidades de saída.



Pedestrian



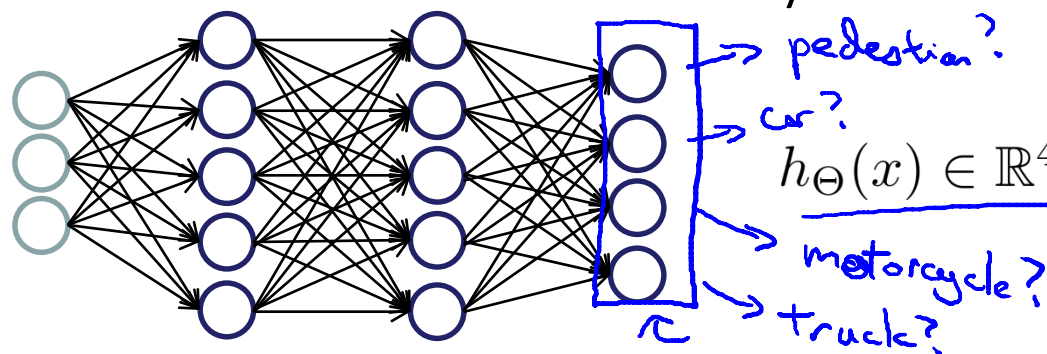
Car



Motorcycle



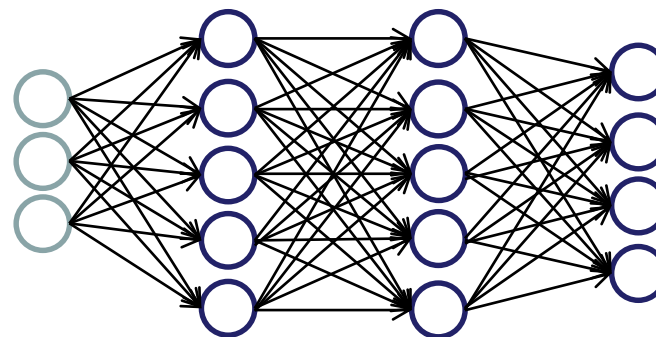
Truck



Queremos $h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, etc.
quando pedestre quando carro quando motocicleta

O Modelo em Camadas Múltiplas

Exemplo: Múltiplas unidades de saída.



$$h_{\Theta}(x) \in \mathbb{R}^4$$

Queremos $h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, etc.
quando pedestre quando carro quando motocicleta

Conjunto de Treinamento: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$

$y^{(i)}$	$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$
	pedestre	carro	motocicleta	caminhão

Aprendizado das Redes MLP

- Os algoritmos de aprendizado para redes MLP são semelhantes ao algoritmo de aprendizado do perceptron.
- Diferença secundária: pode ter várias saídas.
 - Assim, temos um vetor de saída $h_w(x)$ em vez de um único valor.
 - Cada exemplo tem um vetor de saída y .
- Diferença principal: o erro nas camadas ocultas parece misterioso.

Aprendizado das Redes MLP

O ajuste de parâmetros de uma rede neural é realizado a partir de um conjunto treinamento $T = \{(\mathbf{x}(t), \mathbf{v}(t)), t = 1..N\}$, onde, para cada exemplo t :

$\mathbf{x}(t) = (x_1(t), \dots, x_p(t))$ representa o vetor de atributos observados;

- **Em problemas de aproximação de funções:**

$\mathbf{v}(t) = (v_1(t), \dots, v_m(t))$ variáveis de saída observadas em cada registro (variáveis alvo):

- **Em problemas de classificação:**

$\mathbf{v}(t) = (v_1(t), \dots, v_m(t))$ informa a classe correta (observada) associada ao exemplo:

$$v_i(t) = \begin{cases} +1, & \text{se } x(t) \in \omega_i \\ -1, & \text{se } x(t) \notin \omega_i \end{cases}$$

Aprendizado das Redes MLP

A rede deve ser treinada de forma a minimizar o erro entre a saída correta $v(t)$ e saída da rede $y(t) = (y_1(t), \dots, y_m(t))$ através da função de *erro médio quadrático*, calculada como:

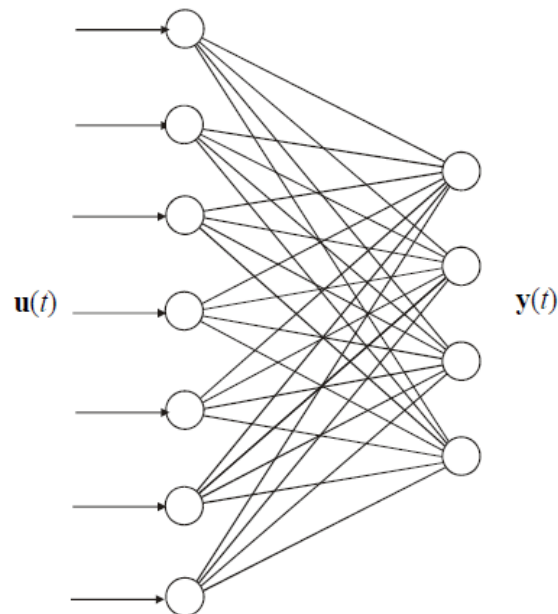
$$J(t) = \frac{1}{2} \sum_{j=1..m} (y_j(t) - v_j(t))^2$$

A saída da RNA é uma função não-linear dos pesos de conexões (considerando-se uma função de ativação não-linear). A estratégia para adaptação dos pesos é considerar o problema como um problema de otimização não linear, cuja solução é calculada através do algoritmo do gradiente.

Aprendizado das Redes MLP

Adaptação da Camada de Saída

O ajuste dos parâmetros da camada de saída é feito como numa rede de duas camadas formada pela camada intermediária $u(t) = (u_1(t), \dots, u_n(t))$ e a camada de saída $y(t) = (y_1(t), \dots, y_m(t))$



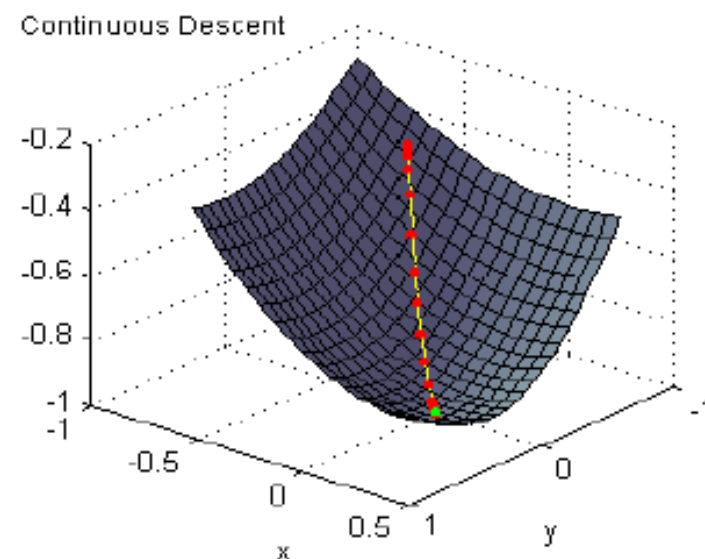
Aprendizado das Redes MLP

O Algoritmo do Gradiente (Steepest Descent)

A cada iteração do método do gradiente, a adaptação dos pesos $\Delta\theta_{ij}$ é proporcional à derivada da função erro em relação a θ_{ij} :

$$\Delta\theta_{ij}(t) = -\eta \frac{\partial J(t)}{\partial \theta_{ij}}$$

onde o parâmetro η é chamado de *taxa de aprendizagem* e determina o quanto o próximo valor do parâmetro θ_{ij} deve “caminhar” na direção contrária da derivada (máxima variação do erro)



Aprendizado das Redes MLP

Cálculo do Gradiente

O cálculo da derivada $\frac{\partial J(t)}{\partial \theta_{ij}}$ é obtido pela regra da cadeia:

$$\frac{\partial J(t)}{\partial \theta_{ij}} = \frac{\partial J(t)}{\partial y_j(t)} \cdot \frac{\partial y_j(t)}{\partial \theta_{ij}} = (y_j(t) - v_j(t)) \frac{\partial y_j(t)}{\partial \theta_{ij}}$$

A saída de cada unidade da rede é calculada como:

$$y_j(t) = f(z(t)) = f\left(\sum_i \theta_{ij} u_i(t) + \beta\right)$$

A derivada $\frac{\partial y_j(t)}{\partial \theta_{ij}}$ é calculada como:

$$\frac{\partial y_j(t)}{\partial \theta_{ij}} = \frac{\partial y_j(t)}{\partial z_j(t)} \frac{\partial z_j(t)}{\partial \theta_{ij}} = f'(z_j(t)) u_i(t)$$

onde $f'(z_j(t))$ é a derivada da função de ativação.

Aprendizado das Redes MLP

Cálculo do Gradiente

A adaptação dos pesos é realizada como:

$$\Delta \theta_{ij}(t) = -\eta \frac{\partial J(t)}{\partial \theta_{ij}} = -\eta \left((y_j(t) - v_j(t)) f'(z_j(t)) u_i(t) \right) = -\eta (\delta_j(t) u_i(t))$$

onde $\delta_j(t) = (y_j(t) - v_j(t)) \cdot f'(z_j(t))$ é chamado de **gradiente local**.

A adaptação dos pesos das conexões *bias* é feita de forma semelhante, considerando-se que as unidades *bias* têm saída constante e igual a 1:

$$\Delta \beta_j(t) = -\eta \delta_j(t) = -\eta (y_j(t) - v_j(t)) f'(z_j(t))$$

Aprendizado das Redes MLP

Cálculo do Gradiente

A versão matricial adaptação dos pesos da matriz de conexões Θ é escrita como:

$$\Delta\Theta(t) = -\eta \left[\frac{\partial J(t)}{\partial \theta_{ij}} \right] = -\eta \begin{bmatrix} \frac{\partial J(t)}{\partial \theta_{11}} & \dots & \frac{\partial J(t)}{\partial \theta_{1m}} \\ \vdots & \ddots & \vdots \\ \frac{\partial J(t)}{\partial \theta_{n1}} & \dots & \frac{\partial J(t)}{\partial \theta_{nm}} \end{bmatrix} = -\eta \begin{bmatrix} u_1(t)\delta_1(t) & \dots & u_1(t)\delta_m(t) \\ \vdots & \ddots & \vdots \\ u_n(t)\delta_1(t) & \dots & u_n(t)\delta_m(t) \end{bmatrix}$$

A adaptação dos pesos das conexões *bias* é escrita como:

$$\Delta\beta(t) = -\eta \left[\frac{\partial J(t)}{\partial \beta_j} \right] = -\eta \begin{bmatrix} \frac{\partial J(t)}{\partial \beta_1} & \dots & \frac{\partial J(t)}{\partial \beta_m} \end{bmatrix} = -\eta [\delta_1(t) \quad \dots \quad \delta_m(t)]$$

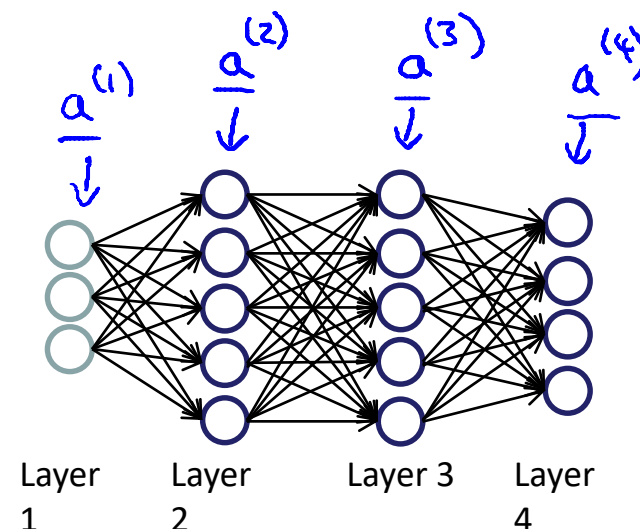
Aprendizado das Redes MLP

Cálculo do Gradiente

Dado um exemplo de treinamento (x, y) :

Propagação para frente:

$$\begin{aligned}
 &\rightarrow \underline{a^{(1)}} = \underline{x} \\
 &\rightarrow z^{(2)} = \Theta^{(1)} a^{(1)} \\
 &\rightarrow a^{(2)} = g(z^{(2)}) \quad (\text{add } \underline{a_0^{(2)}}) \\
 &\rightarrow z^{(3)} = \Theta^{(2)} a^{(2)} \\
 &\rightarrow a^{(3)} = g(z^{(3)}) \quad (\text{add } a_0^{(3)}) \\
 &\rightarrow z^{(4)} = \Theta^{(3)} a^{(3)} \\
 &\rightarrow a^{(4)} = \underline{h_{\Theta}(x)} = g(z^{(4)})
 \end{aligned}$$



Aprendizado das Redes MLP

- Podemos efetuar a **propagação de retorno** do erro da camada de saída para as camadas ocultas.
- O processo de propagação de retorno surge de uma derivação do gradiente de erro global.
- Atualizando as conexões entre as unidades de entrada e as unidades ocultas:
 - É efetuado a propagação de retorno do erro.
 - A idéia é que o nó oculto j é responsável por alguma fração do erro Δ_i em cada um dos nós de saída aos quais ele se conecta.
 - Desse modo, os valores Δ_i são divididos de acordo com a intensidade da conexão entre o nó oculto e o nó de saída e são propagados de volta para fornecer os valores Δ_j referentes à camada oculta.

Aprendizado das Redes MLP

Cálculo do Gradiente: Algoritmo Backpropagation

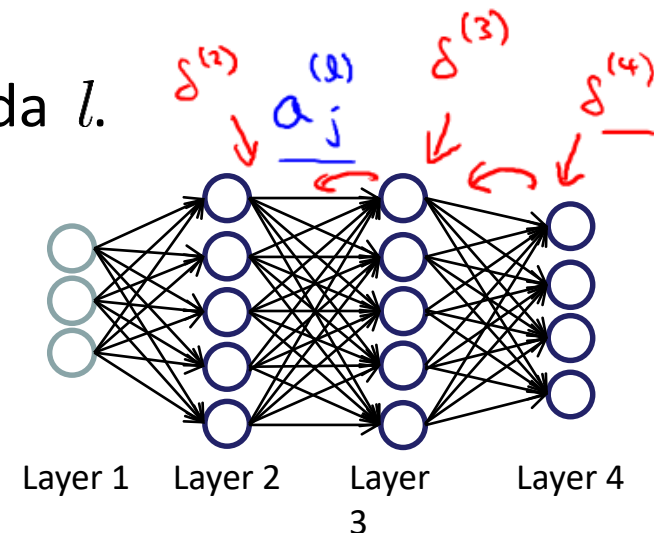
Intuição: $\delta_j^{(l)}$ = “erro” do nó j na camada l .

Para cada unidade de saída (layer $L = 4$)

$$\delta_j^{(4)} = a_j^{(4)} - y_j \quad (h(x))_j \quad \delta_j^{(4)} = a_j^{(4)} - y_j$$

$$\begin{aligned} \delta^{(3)} &= (\Theta^{(3)})^T \delta^{(4)} \cdot \underline{g'(z^{(3)})} \\ \delta^{(2)} &= (\Theta^{(2)})^T \delta^{(3)} \cdot \underline{g'(z^{(2)})} \end{aligned}$$

(No $\delta^{(4)}$)



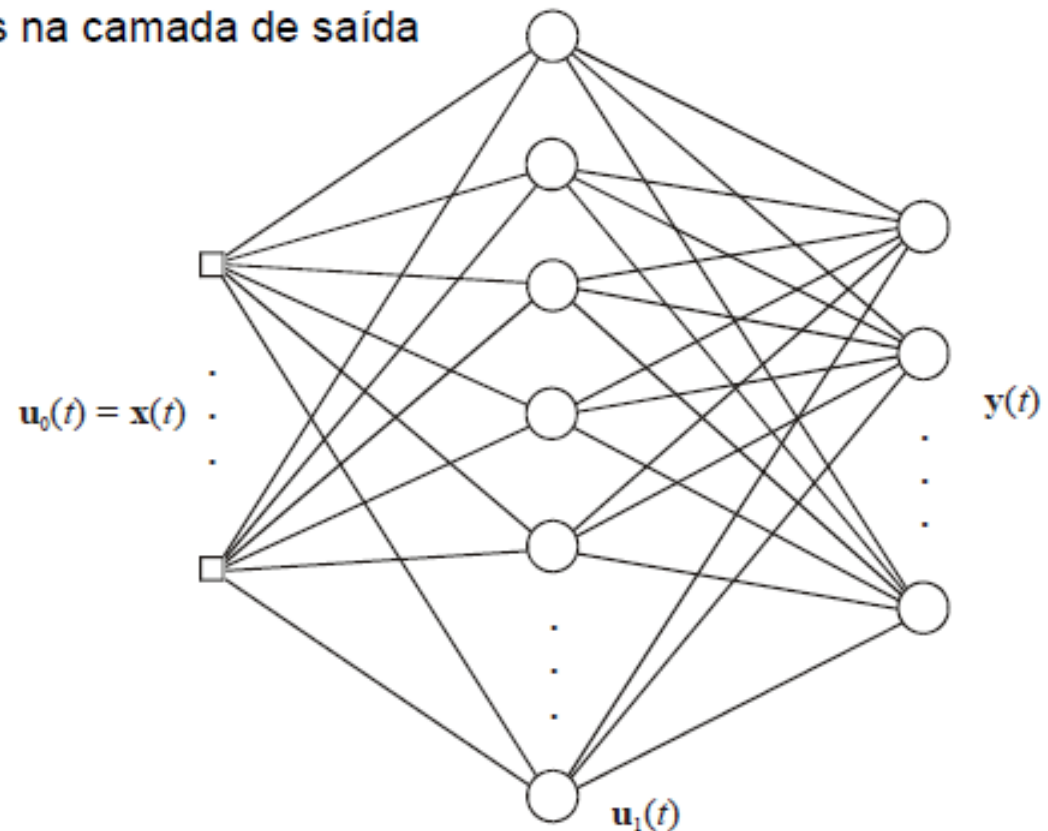
O Algoritmo Backpropagation

Considere a rede MLP $p:n:m$ com p unidades na camada de entrada, n unidades na camada intermediária e m unidades na camada de saída

O processamento da rede a partir de uma entrada $\mathbf{x}(t) = (x_1(t), \dots, x_p(t))$ é calculado como:

$$\mathbf{u}_1(t) = \mathbf{f}(\mathbf{z}_1(t)) = \mathbf{f}(\mathbf{x}(t) \cdot \Theta_0 + \beta_0)$$

$$\mathbf{y}(t) = \mathbf{f}(\mathbf{z}_2(t)) = \mathbf{f}(\mathbf{u}_1(t) \cdot \Theta_1 + \beta_1)$$



O Algoritmo Backpropagation

A adaptação da matriz de conexões Θ_1 , que liga as unidades da camada intermediária às unidades da camada de saída é calculada como:

$$\Delta\Theta_1(t) = -\eta \left[\frac{\partial J(t)}{\partial \theta_{ij}^1} \right] = -\eta \begin{bmatrix} u_1^1(t)\delta_1^2(t) & \dots & u_1^1(t)\delta_m^2(t) \\ \vdots & \ddots & \vdots \\ u_n^1(t)\delta_1^2(t) & \dots & u_n^1(t)\delta_m^2(t) \end{bmatrix}$$

onde o gradiente local $\delta_j^2(t)$ da camada de saída (camada 2) é calculado diretamente pelo erro de saída $\varepsilon_j^2(t) = (y_j(t) - v_j(t))$:

$$\delta_j^2(t) = (y_j(t) - v_j(t)) \cdot f'(z_j^2(t)) = \varepsilon_j^2(t) \cdot f'(z_j^2(t))$$

O Algoritmo Backpropagation

Pelo algoritmo da retropropagação, o gradiente local da camada intermediária $\delta_i^1(t)$ é calculado de forma semelhante como:

$$\delta_j^1(t) = \varepsilon_j^1 \cdot f'(z_j^1(t))$$

onde o erro de saída é “retropropagado” para a camada intermediária, através da matriz de conexão Θ_1 , como:

$$\varepsilon_i^1(t) = \sum_{j=1..m} \delta_j^2(t) \cdot \theta_{ij}^1$$

O que resulta na seguinte fórmula de adaptação dos pesos θ_{ij}^0 :

$$\Delta \theta_{ij}^0(t) = -\eta (\delta_j^1(t) x_i(t)) = -\eta ((\varepsilon_j^1(t) f'(z_j^1(t))) x_i(t)) = -\eta \left(\left(\sum_{j=1..m} \delta_j^2(t) \cdot \theta_{ij}^1 \right) f'(z_j^1(t)) x_i(t) \right)$$

O Algoritmo Backpropagation

Em notação vetorial, a regra de adaptação da matriz Θ_0 é escrita como:

$$\Delta \Theta_0(t) = -\eta \left[\frac{\partial J(t)}{\partial \theta_{ij}^0} \right] = -\eta \begin{bmatrix} x_1(t)\delta_1^1(t) & \dots & x_1(t)\delta_n^1(t) \\ \vdots & \ddots & \vdots \\ x_p(t)\delta_1^1(t) & \dots & x_p(t)\delta_n^1(t) \end{bmatrix}$$

A adaptação do vetor de bias é calculada de forma semelhante como

$$\Delta \beta_0(t) = -\eta \left[\frac{\partial J(t)}{\partial \beta_j} \right] = -\eta [\delta_1^1(t) \quad \dots \quad \delta_n^1(t)]$$

O Algoritmo Backpropagation

O algoritmo pode ser generalizado para a adaptação dos pesos de uma camada genérica k de uma rede de múltiplas camadas.

O erro propagado é calculado em função do gradiente local da camada seguinte:

$$\varepsilon_i^k(t) = \sum_j \delta_j^{k+1}(t) \cdot \theta_{ij}^k$$

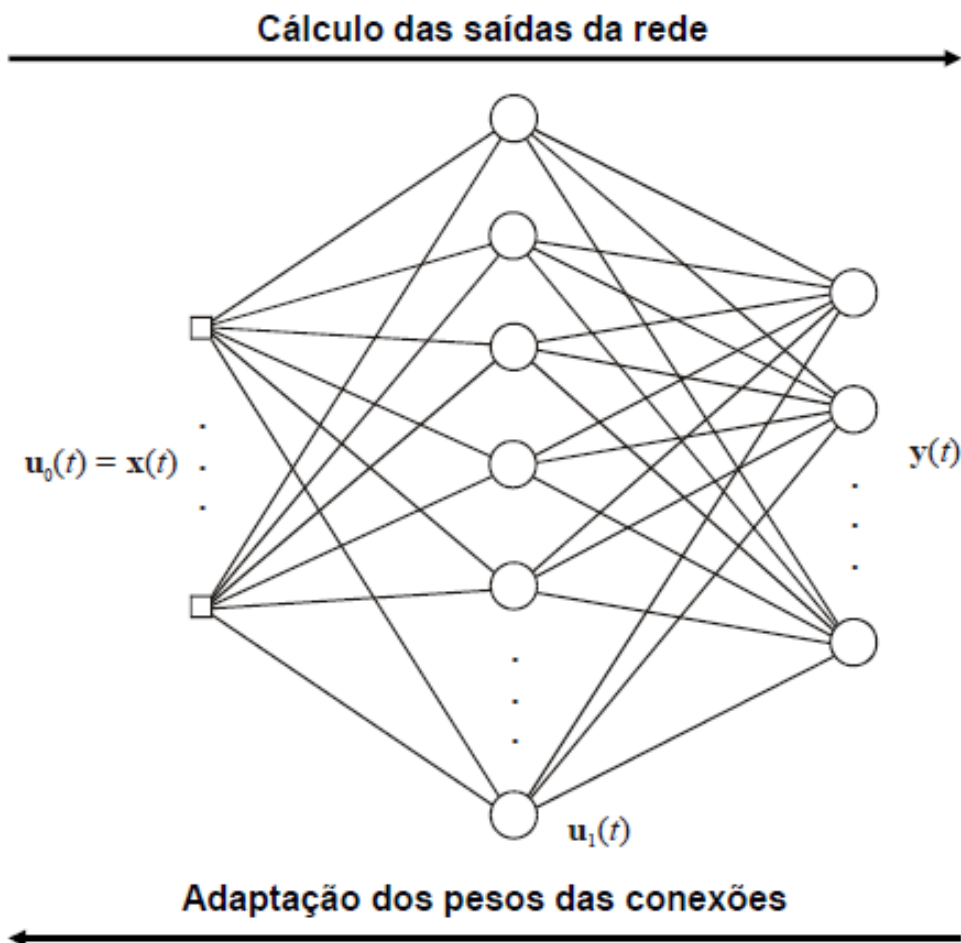
O gradiente local é calculado em função do erro propagado:

$$\delta_j^k(t) = \varepsilon_j^k \cdot f'(z_j^k(t))$$

O ajuste dos parâmetros é calculado em função do gradiente local:

$$\Delta \theta_{ij}^{k-1}(t) = -\eta \left(\delta_j^k u_i^{k-1}(t) \right), \Delta \beta_j^{k-1}(t) = -\eta \delta_j^k$$

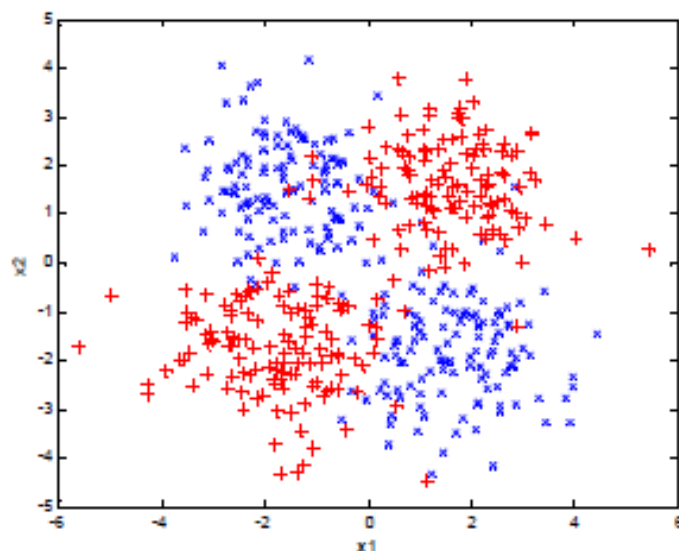
O Modelo em Camadas (resumo)



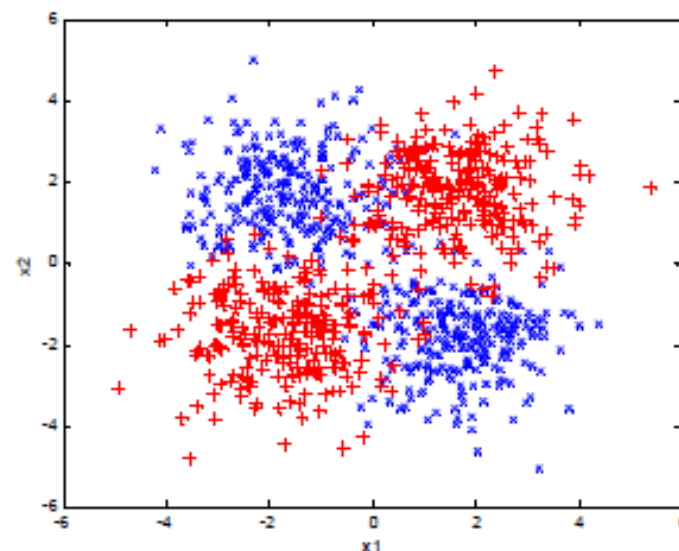
Overfitting

Considere um problema de classificação com duas variáveis de entrada e duas classes cujo conjunto de treinamento contém 500 pontos (250 para cada classe), gerados aleatoriamente. O conjunto de testes, formado por 1000 pontos (500 para cada classe) foi gerado com a mesma distribuição do conjunto de treinamento.

Conjunto de Treinamento



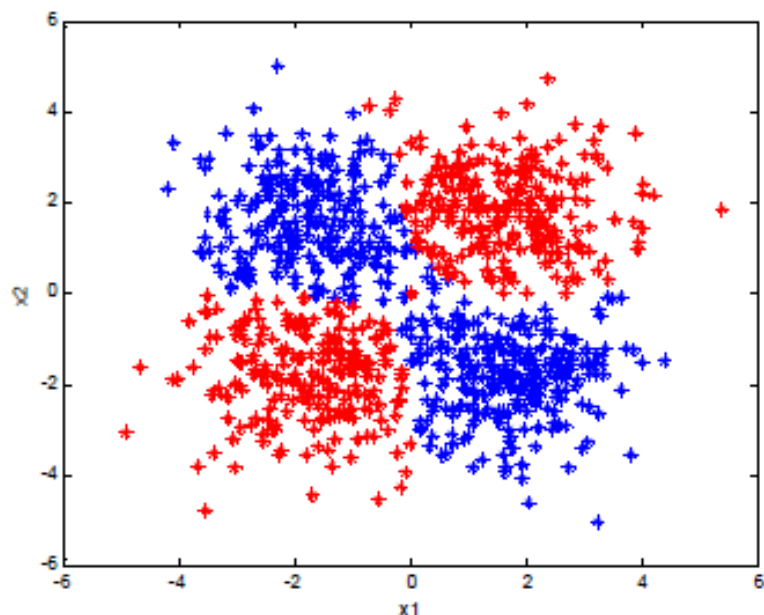
Conjunto de Teste



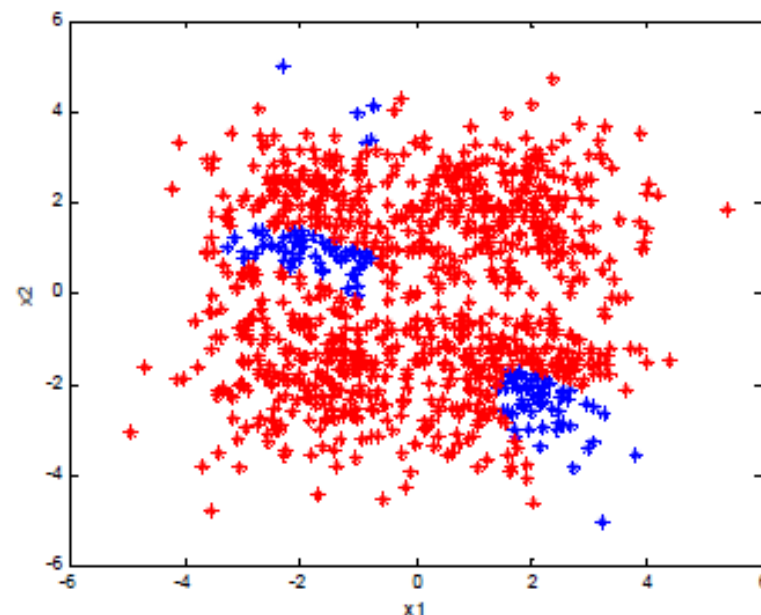
Overfitting

Inicialmente uma rede MLP 2:8:1. O resultado sobre o conjunto de teste apresentou erro total de 12,1%. Para uma topologia 2:256:1, o resultado no conjunto de testes piora sensivelmente (erro de 38,4%).

MLP 2:8:1

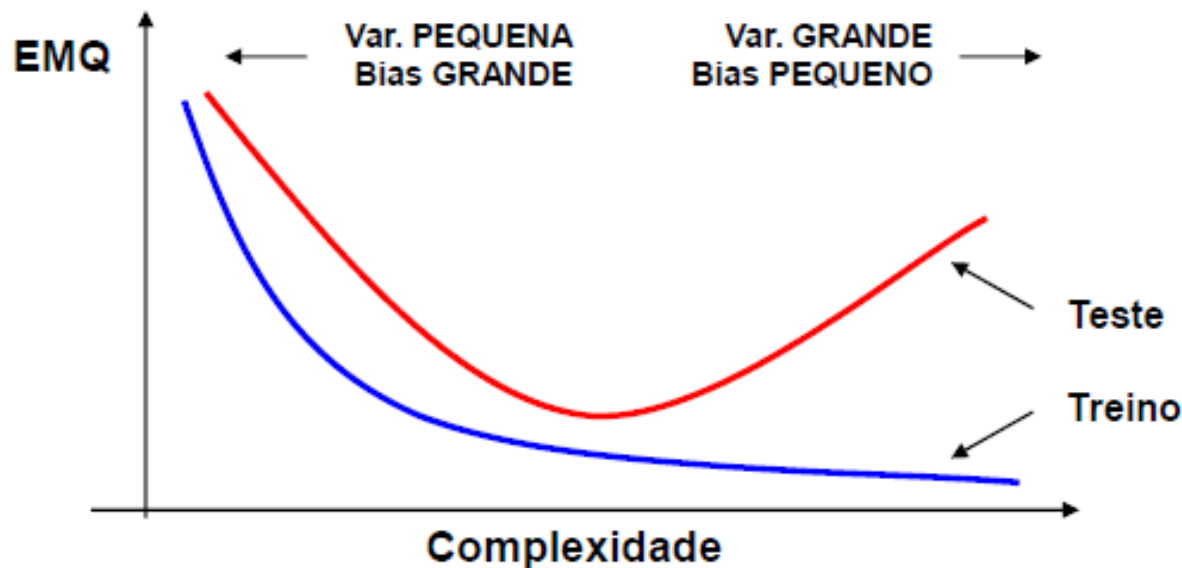


MLP 2:256:1



Bias x Variância

O problema de overfitting está relacionado com a relação Bias x Variância de um modelo ajustado por dados. Quanto maior a complexidade do modelo menor o Bias (no conjunto de treinamento) maior a variância (no conjunto de testes). A solução é buscar uma estrutura capaz de reduzir o erro no conjunto de treinamento quanto mas com boas capacidades de generalização no conjunto de testes.



O Termo de Momento

O termo de momento é uma parcela adicionada ao ajuste de pesos do algoritmo *backpropagation* cujo objetivo é reduzir as oscilações, melhorando a convergência do algoritmo.

$$\Delta \Theta_k(t) = -\eta \left[\frac{\partial J(t)}{\partial \theta_{ij}^k} \right] + \eta_M \underbrace{(\Theta_k(t) - \Theta_k(t-1))}_{\Delta \Theta_k(t-1)}$$

$$\Delta \Theta_k(t) = -\eta \left[\frac{\partial J(t)}{\partial \theta_{ij}^k} \right] + \eta_M \left(-\eta \left[\frac{\partial J(t-1)}{\partial \theta_{ij}^k} \right] \right)$$

onde $0 < \eta_M < 1$ é a taxa de momento.

Estudo de Caso: Previsão de Inadimplência em Telecom

Carlos André Reis Pinheiro

Redes Neurais para Prevenção de Inadimplência em Operadoras de Telefonia

Tese DSc. COPPE/UFRJ – Programa de Eng. Civil, 2005

http://www.coc.ufrj.br/index.php?option=com_content&task=view&id=1289&Itemid=31

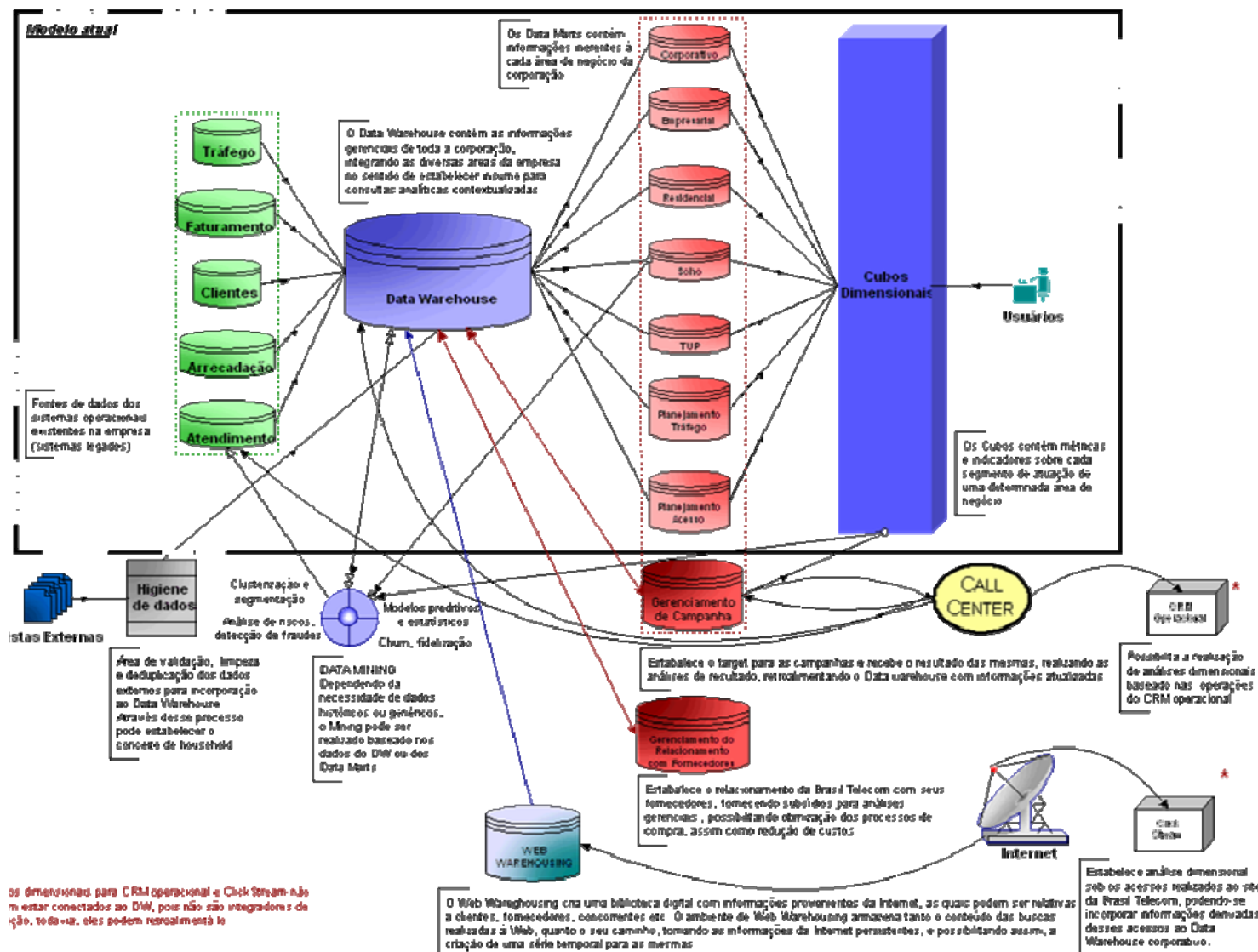
<http://www.acm.org/sigs/sigkdd/explorations/issues/8-1-2006-06/9-Pinheiro.pdf>



Cenário Atual

- As organizações estimam uma perda de receita anual de mais de R\$12 bilhões em todo o mundo por ações de não pagamento.
- As empresas perdem anualmente algo em torno de 7% à 15% do faturamento em eventos de não pagamento.
- Os impactos dos eventos de não pagamento se refletem em diversas áreas da empresa, e afetam todos os clientes, incluindo os bons.
- A taxa de inadimplentes varia entre 25 e 35% da base de clientes ativos.

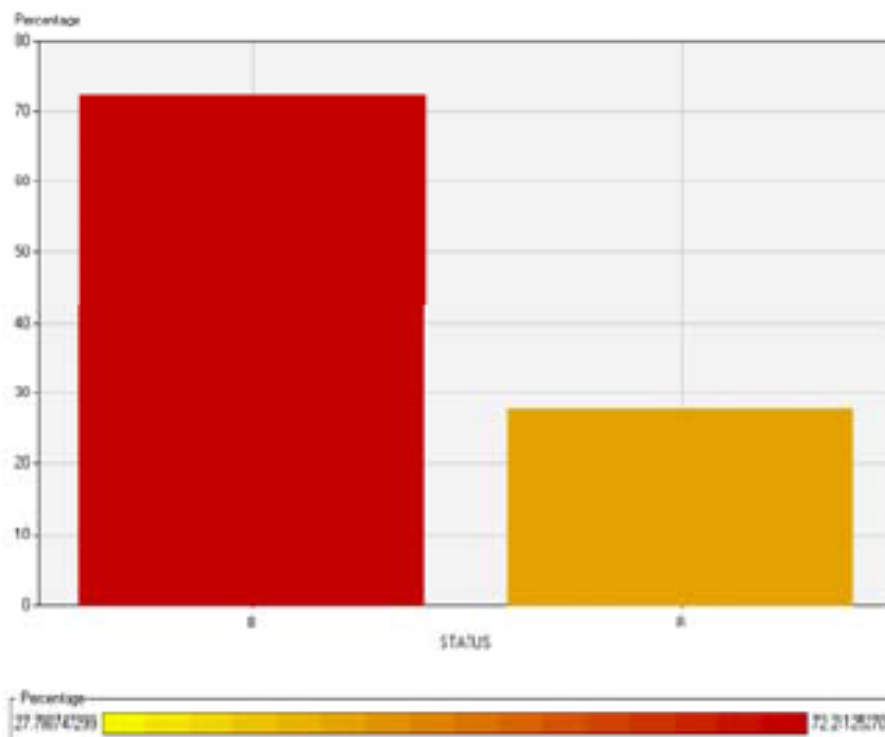
Infra-estrutura em TI da Brasil Telecom



Distribuição dos clientes

Segundo as regras de negócio da empresa, se o cliente ficou mais de 29 dias em atraso, em qualquer um dos meses observados (out/04 à jan/05), ele será considerado ruim, caso contrário, será bom.

A base real possui 72,21% de clientes bons e 27,79% de clientes ruins.



Modelos desenvolvidos:

Modelo de segmentação comportamental para os clientes inadimplentes e modelo de classificação da inadimplência



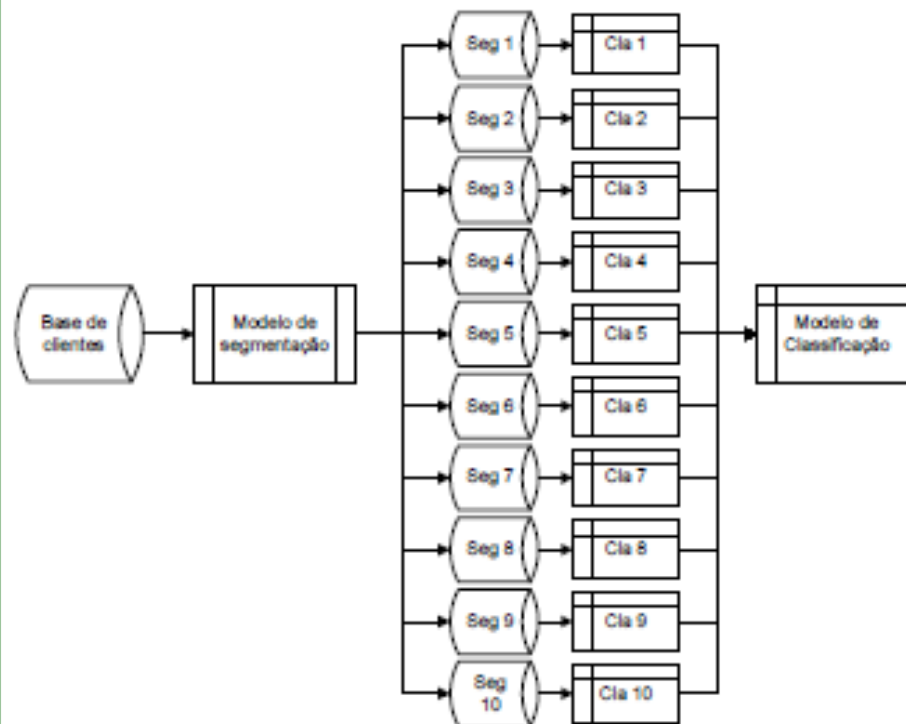
1

Alteração da arquitetura da rede neural para o modelo de classificação da inadimplência



2

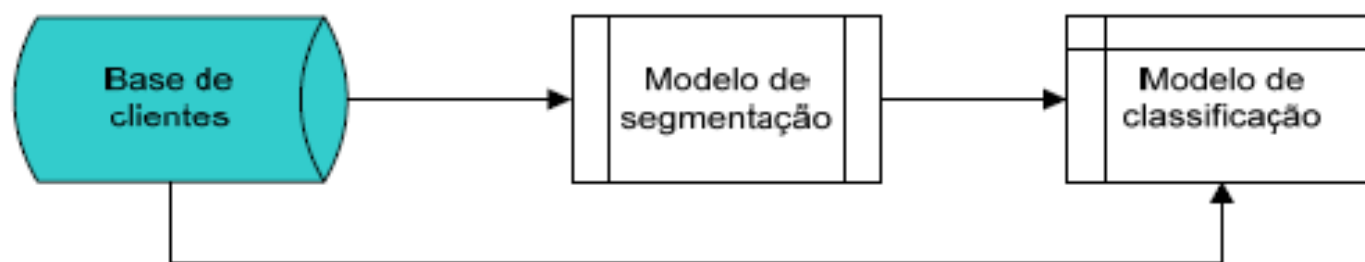
Modelos de classificação da inadimplência com base em um modelo de segmentação comportamental dos clientes



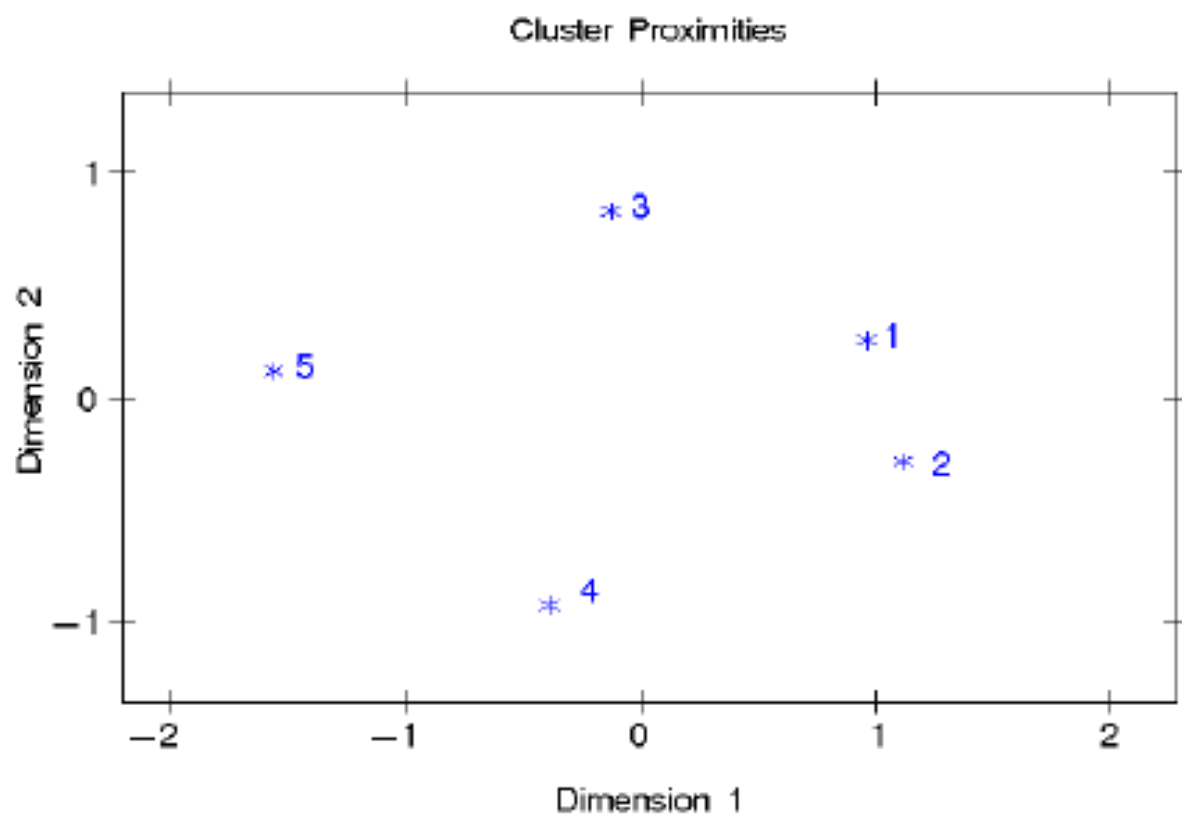
3

Modelo de Segmentação:

Modelo de segmentação comportamental para os clientes inadimplentes e modelo de classificação da inadimplência



Modelo de Segmentação: Identificação de Grupos

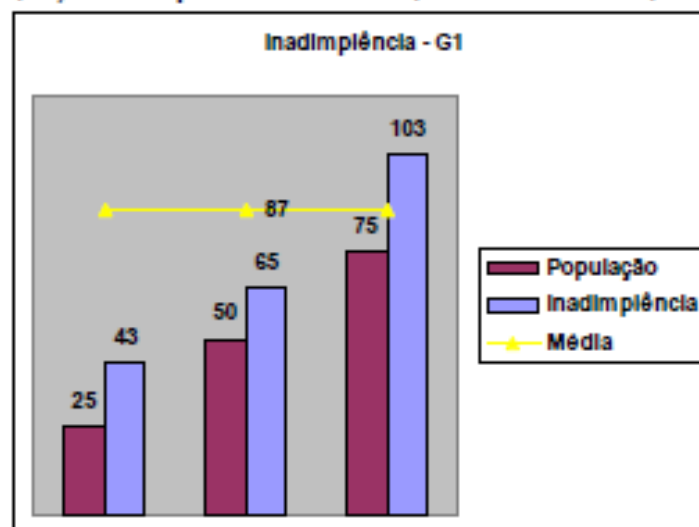


As posições distantes indicam boa distribuição dos grupos entre si, mostrando características distintas e marcantes para cada uma das classes identificadas.

Modelo de Segmentação: Grupo 1

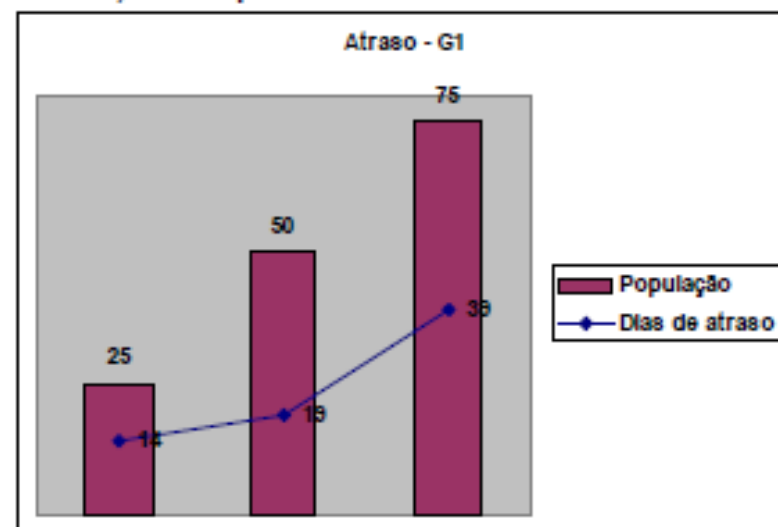
Comportamento da inadimplência por valor

Com média de R\$87, até 25% da população possui média de R\$43, até 50% possui média de R\$65 e até 75% de R\$103.



Comportamento da inadimplência por atraso

Com média de 27 dias, até 25% da população possui média de 14 dias, até 50% possui média de 19 e até 75% de 39.



- Valor de fatura Médio
- Valor de inadimplência Médio
- Período de atraso Alto
- Volume débitos Médio
- Volume de tráfego Médio
- Volume de tráfego local Médio

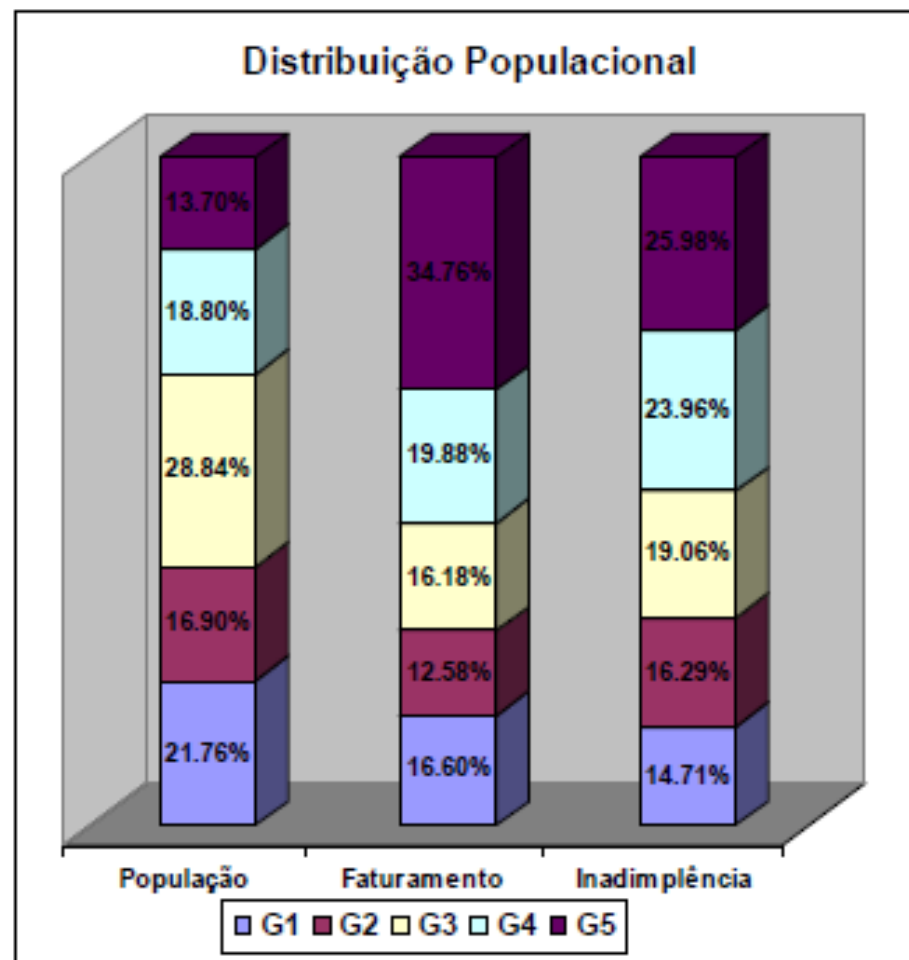
73 minutos
157 pulsos

Classificação:
MODERADO

Análise Comparativa

Distribuição populacional por faturamento e inadimplência

- O grupo 3 possui a maior população.
- O grupo 5 possui a menor população, porém com os maiores valores de faturamento e inadimplência.
- Os grupos 4 e 5 representam 32.4% da população mas respondem por 54.6% do faturamento e 50% da inadimplência.

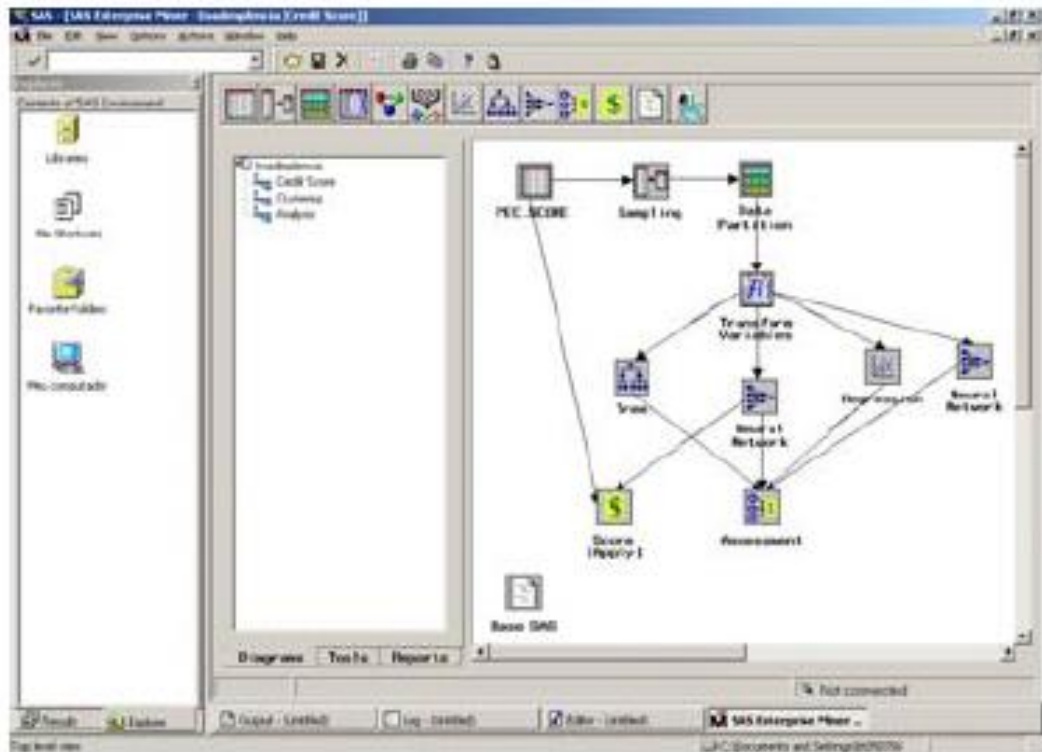
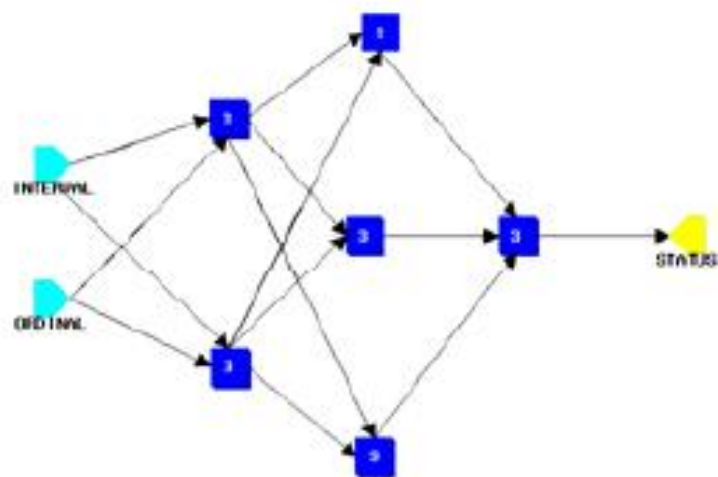


Modelo de Classificação:

Modelo de rede neural MLP para classificação da inadimplência

Classe B = até 29 dias de atraso em todos os mês da janela de observação

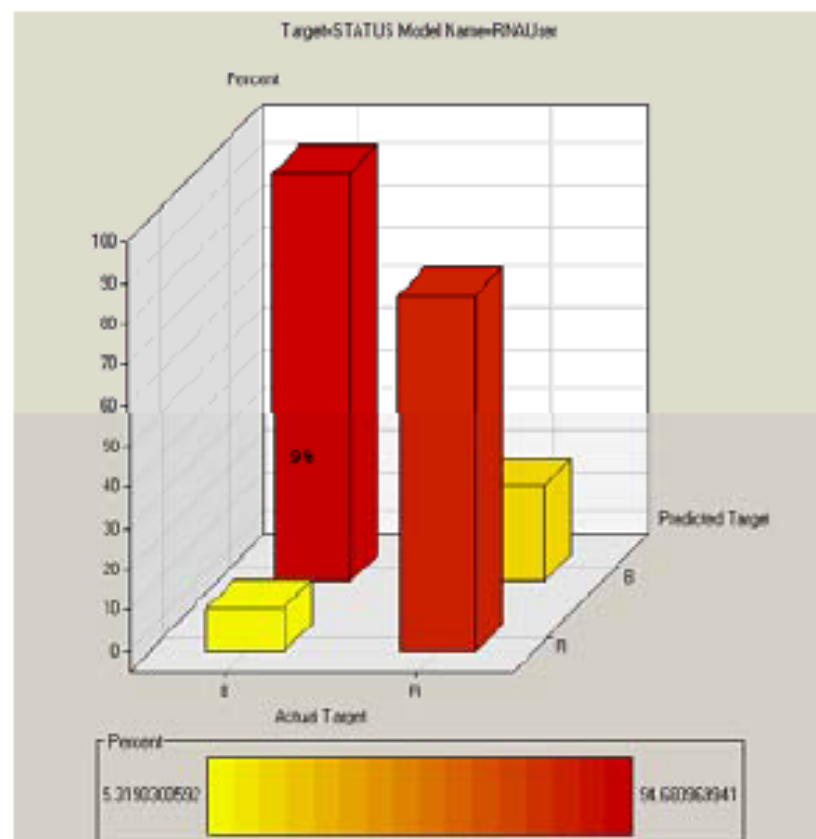
Classe R = 30 dias de atraso em algum mês da janela de observação



Modelo de Classificação 1: Resultados

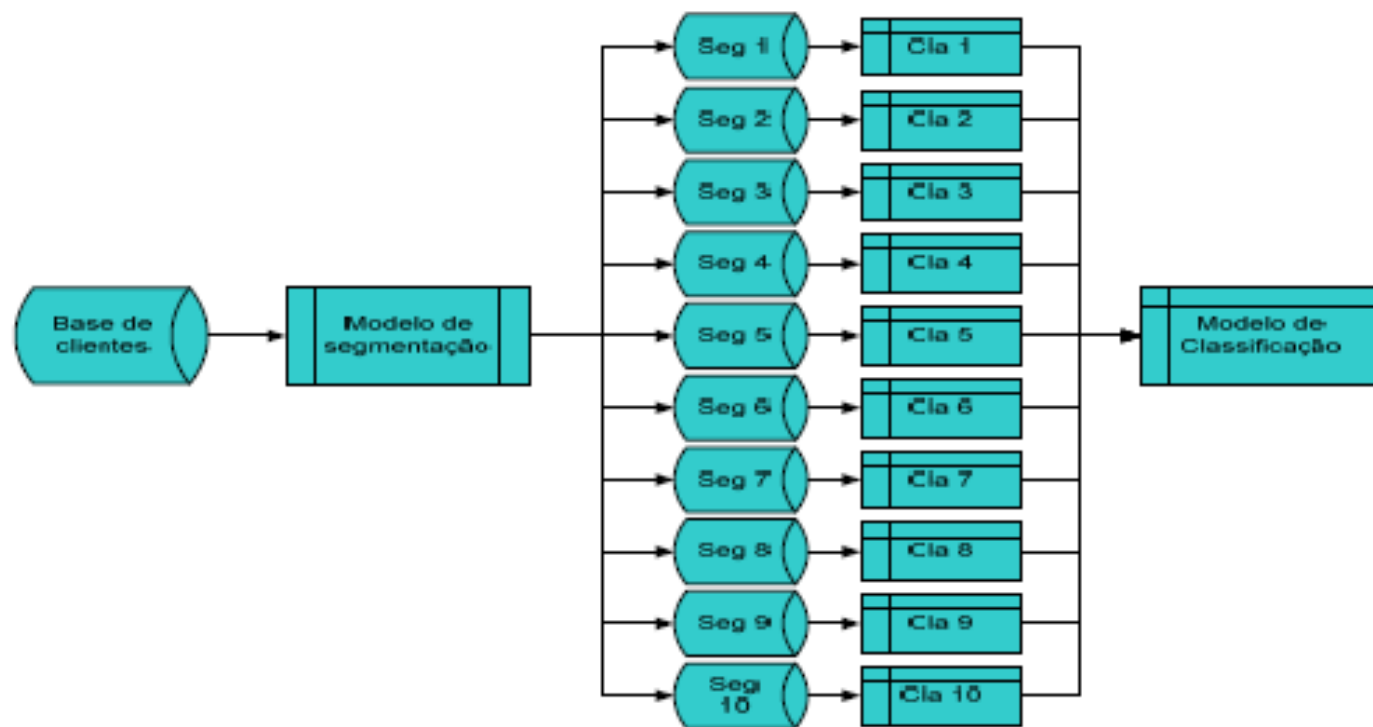
Matriz de confusão para os resultados do modelo de rede neural para classificação da inadimplência

Taxa média de acerto: 82,5%



Modelo de Classificação 2:

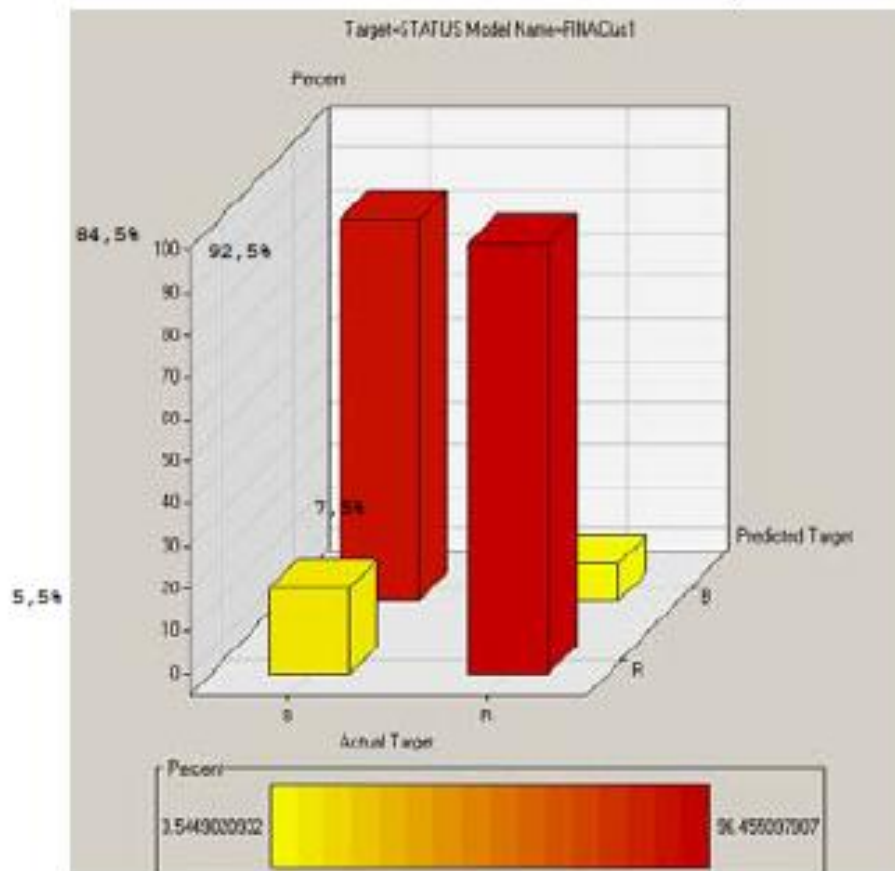
Modelos de classificação da inadimplência com base no modelo de segmentação comportamental de todos os clientes



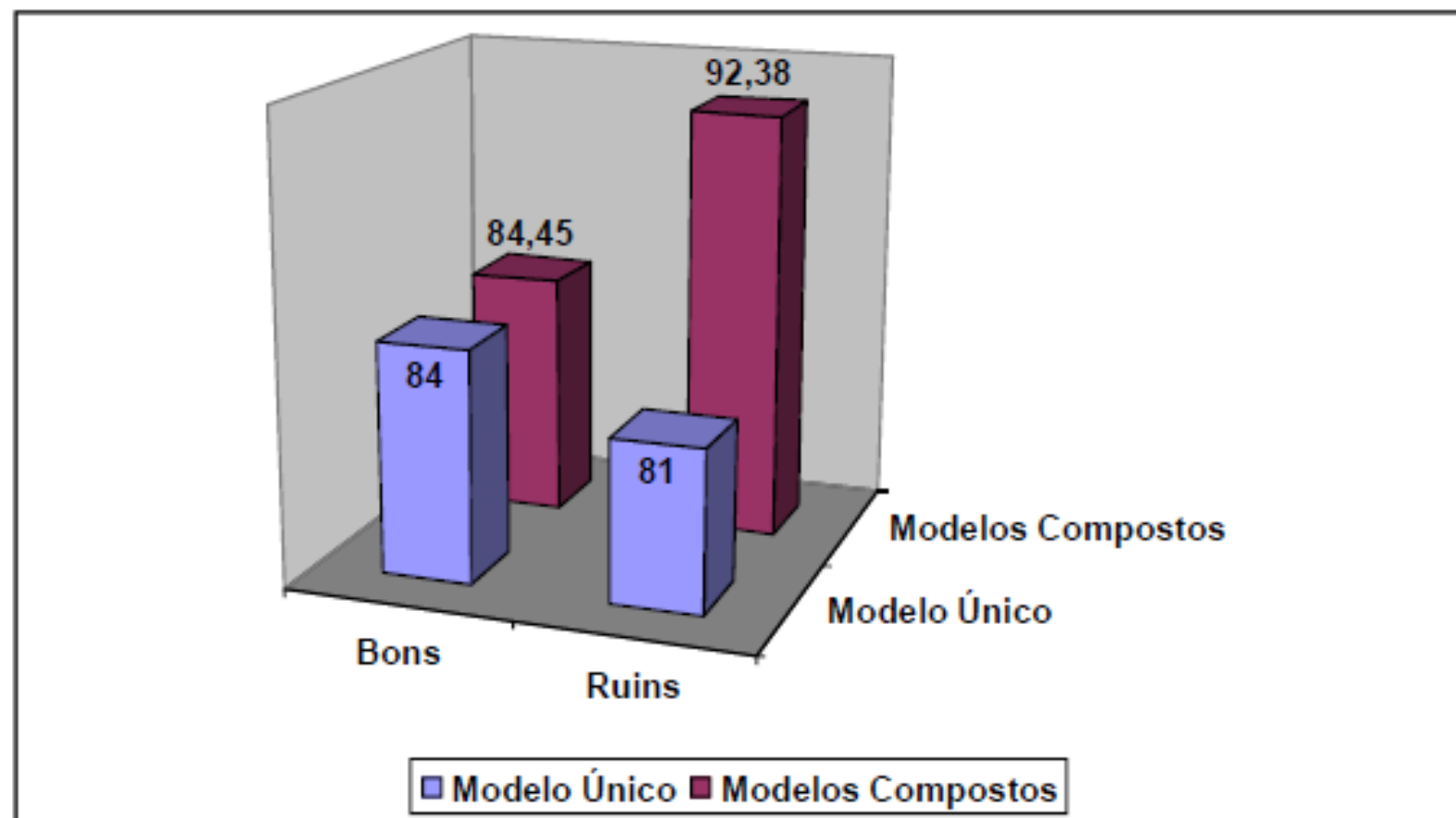
Modelo de Classificação 2: Resultados

Matriz de confusão para os resultados do modelo de rede neural para classificação da inadimplência

Taxa média de acerto: 88,5%



Comparação entre os Modelos:



Análise de Recuperação de Receita:

Economia gerada pela segmentação dos clientes pela não postagem de cartas:

Custos Operacionais	
Cartas de cobrança de 15 dias	R\$ 0,10
Cartas de cobrança de 30 e 60 dias	R\$ 0,05
Postagem	R\$ 0,70
Negativação	R\$ 0,80
Cobrança terceirizada	10 à 20% da fatura
Cobrança judicial	30 à 80% da fatura

Recuperação de Receita			
27,61% dos clientes pagam até 15 dias			
Grupo	Clientes	Débitos	Emissões
G1	74.091	3	222.273
G2	57.543	3	172.629
G3	98.197	3	294.591
G4	64.012	2	128.024
G5	46.647	3	139.941
Total	340.490	-	957.458

Recuperação Total: R\$ 765.966,00

Comparação de Receita x Despesa:

